PSSE36 AGENTIC AI SYSTEM - COMPLETE PRODUCTION-GRADE PRD

Document ID: PRD-PSSE36-AGENT-2026-001
Version: 1.0.0
Date: 2026-05-22
Classification: Engineering-Internal
Status: Draft - Pending Engineering Review

> **Scope:** Agentic AI system for automated power system modelling and single-line diagram generation in PSSE36.
> **Audience:** Staff engineers, system architects, power systems engineers, AI systems developers.
> **Architecture:** Topology-first, IR-driven, orchestrator + specialist agents.
> **Target:** POC in 7-10 weeks with 2-3 engineers. Production evolution over 12-24 months.

§1. Executive Summary

1.1 What This System Is

This document specifies the requirements for an **agentic AI system for automated power system modelling and single-line diagram generation in PSSE36**. The MVP workflow uses structured engineering input (Excel workbooks, CSV, or equivalent tabular schemas) as the canonical deterministic control plane, with a long-term roadmap toward natural language convenience interfaces. The system targets a single software platform — Siemens PTI PSSE version 36 — running on Windows. No other power systems software is in scope.

The system accepts commands such as:

```
"Create a 33kV substation with 2 transformers, double bus arrangement,
6 feeders, CT/PT arrangement and breakers."
```

And automatically:

1. Parses the natural language into a structured electrical intent
2. Constructs an internal topology graph representing the electrical network
3. Validates the topology against electrical engineering rules
4. Computes a layout for visual diagram placement
5. Creates all required PSSE36 objects (buses, branches, machines, loads, transformers)
6. Places those objects onto a PSSE36 single-line diagram with readable coordinates
7. Saves runnable `.sav` and `.raw` files
8. Produces complete execution logs and reasoning traces
9. Self-corrects on failure with bounded retry logic

1.2 Why This Matters

Power system modelling in PSSE is a **high-friction, manually-intensive, error-prone workflow**. Engineers spend hours dragging components, configuring parameters, debugging connectivity, and iterating on diagram layouts. This work is largely deterministic — the same substation configurations are modelled repeatedly across projects. An agentic system that automates this pipeline delivers:

- **4–10x reduction** in model creation time for standard substations
- **Elimination of repetitive CAD-style work** for experienced engineers

- **Dramatically lower onboarding cost** for junior engineers learning PSSE
- **Standardized, reproducible** model generation across teams and projects
- **Full auditability** through structured logs and reasoning traces

1.3 Development Philosophy

POC-first, production-minded. The immediate goal is a working end-to-end demonstration — even if layout is imperfect, assumptions are heuristic, and retry logic is primitive. However, the architecture must be designed so that every subsystem can be incrementally hardened toward production quality without rewriting the foundation.

1.4 Automation Levels

The system is designed to operate across a spectrum of automation:

Level	Mode	Description
L0	Manual	Engineer creates model manually in PSSE (baseline)
L1	Assisted Prompting	Engineer writes structured natural language prompt; system automates topology → model → SLD
L2	Semi-Automated	System partially interprets uploaded drawings/PDFs, proposes topology, engineer confirms
L3	High Automation	System autonomously interprets engineering documents and generates models with minimal intervention

POC target: Level 1. Level 2 is a stretch goal. Level 3 is aspirational and explicitly out of scope for the initial implementation.

Design principle: Engineering reliability > automation purity. If full automation is unreliable, the system intentionally falls back to human-guided workflows.

1.5 System Input Philosophy: Structured-First MVP

"Structured engineering input serves as the canonical source of truth for MVP topology creation."

While the long-term vision supports flexible natural language interfaces (see §2.5), power-system topology definition is inherently structured, exact, and hierarchical. For engineering reliability and deterministic execution in production, the MVP prioritizes explicit, structured, tabular, and engineering-friendly input formats (such as an Excel workbook, CSV, or structured JSON-equivalent schemas) over ambiguous free-form natural language prompting.

This structured engineering input acts as the primary MVP control plane and the direct gateway to the TopologyIR. Free-form natural language prompting (LLM-driven parser) remains fully supported as a future, optional convenience layer positioned upstream of this contract: 1. LLM/NLP (optional convenience layer) parses prompt to generate a structured schema. 2. The structured schema is parsed and validated as a deterministic engineering input. 3. The schema maps into TopologyIR. 4. The system executes deterministic PSSE/SLD generation.

This approach guarantees that the system is built on a highly reliable, testable, and deterministic core first, avoiding the risk of ambiguous topology interpretation in safety-critical power system modeling.

---

§2. Problem Definition

2.1 The Current PSSE Workflow

A power systems engineer creating a substation model in PSSE36 today follows this workflow:

```
Step 1: Open PSSE36 GUI
Step 2: Manually create each bus (set number, name, base kV, type)
Step 3: Manually add transformers (from bus, to bus, winding data, impedance)
Step 4: Manually add lines/branches (from bus, to bus, R, X, B)
Step 5: Manually add generators, loads, switched shunts
Step 6: Manually configure protection elements (breakers, isolators)
Step 7: Open diagram editor
Step 8: Manually drag each bus onto the SLD canvas
Step 9: Manually position transformers, breakers, loads
Step 10: Manually route connections between components
Step 11: Adjust spacing, alignment, labels
Step 12: Run power flow to verify connectivity
Step 13: Debug errors (floating buses, disconnected branches, wrong impedances)
Step 14: Iterate on layout for readability
Step 15: Save .sav file
```

Typical time for a moderately complex substation (double bus, 6 feeders, 2 transformers): 2–6 hours for an experienced engineer, 8–16 hours for a junior engineer.

2.2 Thirteen Core Pain Points

Pain 1: Manual Component Creation

Every bus, branch, transformer, load, and machine must be individually created through PSSE's data entry forms. A 33kV double-bus substation with 6 feeders requires creation of approximately 20–40 individual bus records, 15–30 branch records, and associated transformer data. Each record requires manual entry of 5–15 fields.

Pain 2: Manual Network Modelling

The engineer must mentally translate an engineering drawing into PSSE's internal data model. PSSE uses a bus-branch model where every component is defined by its terminal bus connections. The engineer must assign bus numbers, track from/to bus relationships, and ensure no disconnected islands exist. This mental translation is error-prone, especially for complex topologies.

Pain 3: Manual Topology Creation

Translating a real-world substation arrangement (e.g., breaker-and-a-half, double bus with bus coupler) into PSSE's bus-branch model requires deep understanding of both the electrical topology and PSSE's modelling conventions. Engineers must decide how to represent switching devices, which nodes to model explicitly, and how to handle auxiliary buses.

Pain 4: Repetitive Substation Modelling

Utility engineers model the same substation configurations repeatedly across different projects. A standard 33/11kV distribution substation follows a well-known pattern, yet engineers recreate it from scratch every time. There is no effective template system in PSSE for complex topologies.

Pain 5: Manual Diagram Placement

PSSE's diagram editor requires manual placement of every component on a 2D canvas. Components must be positioned with proper spacing, alignment, and routing. There is no auto-layout feature. Engineers spend 30–50% of total modelling time on diagram arrangement.

Pain 6: Junior Engineer Onboarding

PSSE has a steep learning curve. Junior engineers require 3–6 months to become proficient at model creation. During this period, they produce frequent errors: wrong bus connections, missing impedance data, floating equipment, corrupted save files. Senior engineer review time is substantial.

Pain 7: Knowledge Bottleneck

Substation topology knowledge is concentrated in senior engineers. How to model a breaker-and-a-half arrangement, how to represent CT/PT chains, how to handle bus section switches — this knowledge is tribal and rarely documented. When senior engineers leave, institutional knowledge is lost.

Pain 8: High Modelling Time

End-to-end model creation for a medium-complexity system (50–200 buses) takes 1–4 days of engineering time. For large systems (500+ buses), this extends to weeks. Most of this time is spent on repetitive, deterministic tasks that could be automated.

Pain 9: Repeated Engineering Work

Across an organization, the same substation patterns are modelled independently by different engineers. No systematic reuse mechanism exists. A 33kV distribution substation modelled for Project A provides zero benefit when modelling the identical topology for Project B.

Pain 10: Diagram Editing Friction

When the electrical model changes (e.g., adding a feeder, changing bus configuration), the SLD must be manually re-arranged. There is no automatic re-layout. Component positions become stale, connections overlap, and the diagram becomes unreadable. Engineers spend significant time on cosmetic fixes.

Pain 11: CAD-Style Repetitive Work

Much of PSSE modelling is functionally identical to CAD drafting — placing symbols, routing connections, adjusting spacing. This work requires engineering knowledge to set up correctly but is fundamentally mechanical once the topology is defined.

Pain 12: Topology Debugging

Connectivity errors in PSSE manifest as power flow convergence failures, islanded buses, or nonsensical results. Debugging requires manually tracing bus connections through data tables. There is no visual topology validation tool. Engineers often resort to printing bus-branch lists and manually checking connectivity.

Pain 13: Human Error in Repetitive Tasks

Manual data entry across dozens of bus records leads to typographical errors: wrong bus numbers in branch definitions, swapped from/to buses, incorrect base voltages, wrong impedance units. These errors are difficult to detect and can propagate through studies.

2.3 Why SLD Generation Is Fundamentally Difficult

Single-line diagram generation is not a trivial rendering problem. It is a **constrained graph layout problem** with domain-specific aesthetic requirements:

1. **Topological correctness:** Every electrical connection must be represented as a visual connection. No spurious or missing connections are permissible.
2. **Engineering conventions:** SLDs follow strict visual conventions. Busbars are horizontal lines. Feeders drop vertically. Transformers are positioned between voltage levels. Breakers and isolators are placed in sequence. These conventions are culturally embedded in power engineering practice and cannot be violated without confusing the reader.
3. **Readability constraints:** Components must not overlap. Connection lines must not cross unnecessarily. Labels must be readable. Spacing must be uniform. These are non-trivial constraint satisfaction problems.
4. **Hierarchical structure:** Real substations have a natural hierarchy: voltage level → bus arrangement → bay → equipment chain. The SLD layout must reflect this hierarchy visually.
5. **Scaling:** Layout algorithms that work for 5 buses may fail catastrophically for 50 buses. The algorithm must degrade gracefully as network size increases.
6. **Aesthetic quality:** Engineers are trained to read well-formatted SLDs. A poorly laid out diagram is not merely ugly — it is functionally impaired because it obscures the topology it is meant to communicate.

2.4 Why Diagram Generation Is a Graph Problem

An SLD is a visual projection of an electrical graph G(V, E) onto a 2D plane, subject to:

- **V → (x, y) coordinate mapping** for each electrical entity
- **E → polyline routing** for each connection
- **Constraint set C** defining minimum spacing, alignment rules, hierarchy preservation, and crossing minimization

This is a well-studied class of problem in graph drawing literature (Sugiyama layered layout, orthogonal grid layout, hierarchical layout). The key insight is that **SLD generation is deterministic geometry constrained by engineering conventions**, not free-form artistic layout. This makes it amenable to algorithmic solution.

2.5 Why Natural Language Interfaces Matter Here

Power systems engineers describe systems in natural language habitually:

- "*Double bus arrangement with bus coupler*"
- "*Six outgoing 11kV feeders*"
- "*132/33kV transformation with two 50MVA transformers*"

This language is semi-structured. Voltage levels, equipment counts, topology types, and arrangement patterns are conveyed in compact phrases that have precise engineering meaning. A well-designed NLP layer can extract deterministic topology specifications from this language with high reliability because the domain vocabulary is constrained (perhaps 200–500 relevant terms) and the grammar of equipment descriptions is relatively regular.

2.6 Quantified Impact

Metric	Manual Workflow	With System (L1)	Improvement
Simple substation creation	2–4 hours	5–15 minutes	8–16x
Complex substation creation	6–16 hours	15–45 minutes	8–20x
Junior engineer onboarding	3–6 months	2–4 weeks	4–6x
Topology errors per model	2–8	0–1 (validated)	~90% reduction
Diagram re-layout after change	30–90 minutes	Automatic	~100% reduction
Cross-project reuse	None	Template-based	New capability

---

§3. Product Vision

3.1 Core Vision Statement

"AI-assisted electrical CAD automation for PSSE36."

The system transforms power system modelling from a manual CAD workflow into an engineering-input-driven workflow, where the engineer specifies *what* they want — via structured engineering data (Excel / CSV) for MVP, or via natural language prompts in future releases — and the system deterministically builds it in PSSE.

3.2 Evolution Path

The system evolves through four maturity stages. Each stage is a superset of the previous:

```
Stage 1: ENGINEERING ASSISTANT (POC)
├── Accepts structured natural language prompts
├── Generates topology from prompt
├── Creates PSSE model automatically
├── Places components on SLD with heuristic layout
├── Saves runnable files
├── Logs all actions
└── Target: 3-6 months

Stage 2: AUTOMATION TOOL (v1.0)
├── Everything in Stage 1, plus:
├── Handles ambiguous prompts with clarification
├── Validates topology against engineering rules
├── Retries on failure with bounded self-correction
├── Human approval checkpoints
├── Template library for common substations
└── Target: 6-12 months

Stage 3: SEMI-AUTONOMOUS MODELLER (v2.0)
├── Everything in Stage 2, plus:
├── Partially interprets uploaded SLD PDFs
├── Suggests topology from partial specifications
├── Learns from engineer corrections
├── Multi-substation network composition
├── Layout quality approaching hand-drawn standards
└── Target: 12-24 months

Stage 4: ENGINEERING COPILOT (v3.0+)
├── Everything in Stage 3, plus:
├── Autonomous interpretation of engineering documents
├── Interactive model editing via conversation
├── Integration with study workflows (power flow, fault analysis)
├── Team collaboration features
├── Enterprise deployment
└── Target: 24-36 months
```

3.3 What This Is NOT

This system is **not**:

- A general-purpose AI assistant that happens to know about power systems
- A replacement for engineering judgment on protection coordination, stability, or planning
- A real-time SCADA interface
- A multi-tool orchestrator that abstracts across PSSE, PowerWorld, and DigSilent
- A cloud-native SaaS platform
- An AGI that understands all of electrical engineering

It is a **narrowly-scoped, deeply-integrated automation tool** for one specific workflow (model creation + SLD generation) in one specific tool (PSSE36).

---

§4. Scope Definition

4.1 In-Scope (Must Have for POC)

ID	Requirement	Description
S-01	PSSE36 target	System operates exclusively with PSSE version 36 on Windows
S-02	Structured engineering input	System accepts row-wise structured engineering data (Excel / CSV / JSON) defining electrical topology with explicit connectivity. Natural language input is a future convenience layer.
S-03	Topology generation	System constructs an electrical topology graph from parsed intent
S-04	Bus creation	System automatically creates PSSE bus records with appropriate numbering, naming, base kV
S-05	Branch creation	System creates branches (lines, transformers) with from/to bus connections
S-06	Transformer modelling	System creates two-winding transformers with winding data
S-07	Load/generator stubs	System creates placeholder loads and generators where specified
S-08	Breaker representation	System models breakers as zero-impedance branches or bus-tie connections
S-09	SLD auto-placement	System assigns (x, y) coordinates to all components for PSSE diagram
S-10	File persistence	System saves `.sav` and/or `.raw` files that load in PSSE without errors
S-11	Execution logging	Every action is logged with timestamp, agent, input/output, and result
S-12	Retry on failure	System retries failed PSSE operations up to a configurable bound
S-13	Human approval	System pauses for engineer confirmation before PSSE execution
S-14	IR generation	System produces a JSON intermediate representation of the topology
S-15	Validation	System checks topology graph for basic electrical consistency before execution

4.2 In-Scope (Stretch Goals for POC)

ID	Requirement	Description
S-16	CT/PT representation	Model current and potential transformers in the topology
S-17	Isolator/disconnect modelling	Model isolators as switching devices in the breaker arrangement
S-18	Bus coupler	Support bus coupler topology in double-bus arrangements
S-19	Template substations	Pre-built templates for common substation configurations
S-20	Layout quality scoring	Score generated layouts against readability metrics

4.3 Explicitly Out of Scope

ID	Non-Goal	Rationale
N-01	Multi-software support	Architectural complexity; PSSE36 integration alone is substantial
N-02	Cloud deployment	Unnecessary for POC; PSSE requires local Windows installation
N-03	Enterprise authentication	No multi-user requirement in POC
N-04	Multi-user collaboration	Single-engineer workflow for POC
N-05	DigSilent/PowerWorld/ETAP	Violates core constraint C1
N-06	Real-time SCADA integration	Different problem domain entirely
N-07	Protection coordination studies	Requires additional analysis engines beyond model creation
N-08	Optimal power flow / market simulation	Out of scope for model creation tool
N-09	Digital twin functionality	Requires real-time data integration
N-10	Reinforcement learning	Unnecessary complexity; rule-based approaches sufficient for POC
N-11	Custom LLM training/fine-tuning	Off-the-shelf models with structured prompting are sufficient
N-12	GPU compute dependency	System must run on standard engineering workstations
N-13	Multi-agent swarm architectures	Disciplined few-agent architecture preferred
N-14	Vendor-neutral abstraction layer	Premature abstraction; optimize for PSSE36 directly
N-15	PDF/drawing auto-interpretation	Level 3 automation; not required for POC (Level 1)
N-16	Three-winding transformers	Edge case; two-winding covers 80%+ of use cases
N-17	HVDC modelling	Specialized; not required for standard substation modelling
N-18	Dynamic simulation setup	Different workflow; focus is on steady-state model creation

ID	Non-Goal	Rationale
N-19	Free-form natural language topology inference	The MVP prioritizes deterministic structured engineering input. The system will not rely on ambiguous prompt interpretation to create electrical topologies; LLM-driven topology generation remains downstream/future work.

4.4 Scope Freeze Recommendation

For the POC, scope should be frozen at:

"A system that accepts a structured engineering input (Excel / CSV / JSON schema) or a structured natural language description of a substation or simple network (≤50 buses), generates a valid PSSE36 model with automatic SLD placement, and saves runnable files — with full logging and human approval before execution."

Note on input modality: The primary interface for topology creation in the MVP is the validated, deterministic row-wise structured engineering schema. Prompt-based natural language interaction remains an optional convenience feature.

Any feature request beyond this boundary should be deferred to post-POC unless it is blocking for the demo.

---

§5. Non-Goals (Expanded Rationale)

Beyond the scope exclusion table in §4.3, the following architectural non-goals require explicit justification because they represent common over-engineering traps:

5.1 No Vendor-Neutral Abstraction Layer

Anti-pattern: Building an abstract `PowerSystemTool` interface that could theoretically support PSSE, PowerWorld, DigSilent, etc.

Why this is wrong for POC: PSSE36's API surface (psspy), data model (bus-branch), file formats (.sav/.raw), and diagram system are deeply specific. An abstraction layer would: - Add 2–4 weeks of engineering effort for zero immediate value - Force lowest-common-denominator API design that cannot exploit PSSE-specific features - Create maintenance burden for an abstraction used by exactly one backend - Encourage premature generalization before the PSSE integration is even proven

Correct approach: Build directly against PSSE36. If multi-tool support is ever needed (unlikely for 12+ months), extract the abstraction from the working PSSE implementation.

5.2 No Kubernetes / Container Orchestration

Why: PSSE36 is a Windows desktop application with a COM/Python bridge. It cannot run in a Linux container. The system must run on the same Windows machine where PSSE36 is installed. Container orchestration adds zero value and substantial complexity.

5.3 No Distributed Microservices

Why: The system has one user, one PSSE instance, one execution pipeline. There is no scaling requirement. A monolithic Python application with clean module boundaries is the correct architecture. Microservices would add network serialization overhead, deployment complexity, and debugging difficulty for zero benefit.

5.4 No Custom LLM Training

Why: The domain vocabulary for substation description is small (~200–500 terms). Structured prompting with few-shot examples against a frontier model (GPT-4o, Claude 3.5 Sonnet) achieves sufficient extraction accuracy. Fine-tuning requires training data that doesn't exist yet. The correct strategy is: use structured prompting → collect correction data from human approvals → evaluate fine-tuning need after 100+ real usage sessions.

5.5 No Real-Time Processing Requirements

Why: Model creation is a batch workflow. The engineer submits a prompt, waits for execution (seconds to low minutes), reviews the result. There are no latency SLAs below "reasonable interactive response" (~30 seconds for simple, ~2 minutes for complex). Do not optimize for real-time performance.

---

§6. User Personas

6.1 Persona 1: Junior Power Systems Engineer

Profile: 0–3 years experience. Recently graduated with electrical engineering degree. Assigned to power system studies team. Learning PSSE.

Current Pain Points: - PSSE GUI is intimidating. Menu structure is deep and unintuitive - Doesn't know PSSE conventions: bus numbering schemes, impedance units, winding data formats - Makes frequent connectivity errors: wrong from/to bus assignments, floating buses - Spends 2–3x longer than senior engineers on identical tasks - Afraid to ask senior engineers for help on "basic" tasks repeatedly - Cannot debug power flow convergence failures caused by modelling errors - Diagram placement takes hours of manual drag-and-drop

Workflow with System: 1. Describes desired network in natural language (learned from engineering courses) 2. System generates model and presents assumptions for approval 3. Junior engineer reviews assumptions (learning opportunity) 4. System creates PSSE model and SLD automatically 5. Junior engineer runs power flow study on the pre-built model

Benefits: - Reduces PSSE learning curve from months to weeks - Eliminates manual data entry errors - Provides implicit education through assumption display - Builds confidence through successful model creation

Risk Tolerance: Low. Wants confirmation at every step. Prefers verbose explanations.

Expected UX: Chat-style interface with detailed explanations. "Here's what I'm going to do and why." Progress indicators. Clear error messages with suggested fixes.

6.2 Persona 2: Senior Power Systems Engineer

Profile: 10+ years experience. Expert PSSE user. Creates complex network models routinely. Leads study teams.

Current Pain Points: - Spends 40–60% of time on repetitive model creation that doesn't require expertise - Could be doing higher-value work (analysis, planning, mentoring) - Frustrated by PSSE's lack of automation for well-understood patterns - Must manually recreate similar substations across projects - Diagram re-layout after model changes is tedious - Reviewing junior engineers' models consumes significant time

Workflow with System: 1. Issues terse, expert-level commands: "33/11kV double bus, 6 feeders, bus coupler, 2x50MVA" 2. Quickly approves assumptions (or overrides with specific values) 3. Reviews generated model in PSSE 4. Makes minor manual adjustments if needed 5. Proceeds directly to analysis

Benefits: - Recovers 2–4 hours per substation model - Reduces context-switching between design thinking and data entry - Standardizes output quality across team members - Templates capture tribal knowledge for team reuse

Risk Tolerance: High. Willing to accept imperfect automation and manually fix edge cases. Values speed over hand-holding.

Expected UX: Minimal friction. Terse prompts accepted. Skip confirmations for standard patterns. Show diffs, not explanations.

6.3 Persona 3: Utility Planning Engineer

Profile: Works at electric utility. Responsible for system planning studies. Creates models of future substation configurations for planning scenarios.

Current Pain Points: - Must model dozens of substation variants for scenario analysis - Consistency across models is critical but hard to enforce - Model review and QA is time-consuming - Different engineers model the same substation differently - Documentation of modelling assumptions is inconsistent

Workflow with System: 1. Defines substation specification from planning documents 2. Uses consistent prompt templates across scenarios 3. System generates standardized models with identical conventions 4. All assumptions are automatically documented in logs 5. Runs batch scenarios on generated models

Benefits: - Guaranteed consistency across scenario models - Automatic documentation of all assumptions - Rapid scenario generation (minutes instead of days) - Auditable model creation process

Risk Tolerance: Medium. Requires validation but values standardization highly.

Expected UX: Template-driven. Structured input forms. Batch generation. Export of assumption logs for regulatory documentation.

6.4 Persona 4: EPC Engineer

Profile: Works at Engineering, Procurement, and Construction firm. Creates PSSE models for substation design packages. Tight project deadlines.

Current Pain Points: - Project timelines demand rapid model creation - Must produce models for client review on aggressive schedules - Model quality must meet client utility standards - Diagram aesthetics matter for client deliverables - Repetitive modelling across multiple project sites

Workflow with System: 1. Receives substation specification from design team 2. Translates specification into natural language prompt 3. System generates model and diagram 4. Engineer reviews and adjusts for client-specific requirements 5. Delivers model as part of design package

Benefits: - Meets tight project deadlines - Consistent quality across project deliverables - Reduces modelling cost per project - Frees engineering time for design optimization

Risk Tolerance: Medium-high. Needs reliable output but accepts manual cleanup.

Expected UX: Fast execution. Clean diagram output. Easy parameter override. Export-ready file formats.

6.5 Persona 5: Research Engineer / Academic

Profile: University researcher or R&D engineer. Builds test networks for algorithm validation, protection studies, or stability analysis.

Current Pain Points: - Needs many small test networks quickly - Cares about topology correctness, not diagram aesthetics - Wants reproducible network generation - Manual PSSE model creation is a bottleneck for research velocity - Often needs parametric variations of the same base topology

Workflow with System: 1. Describes test network requirements programmatically 2. System generates base model 3. Researcher modifies parameters for study variations 4. Uses generated models as inputs to analysis scripts

Benefits: - Rapid test case generation - Reproducible network creation from version-controlled prompts - Focus on research instead of model building - Easy parametric variation through prompt modification

Risk Tolerance: Highest. Accepts rough layouts. Cares about electrical correctness.

Expected UX: CLI/script interface. Programmatic access. Batch generation. JSON output for post-processing.

---

§7. Functional Requirements

7.1 Core Functional Requirements

FR-01: Natural Language Command Acceptance

- **Description:** System shall accept free-text English commands describing electrical networks, substations, or system modifications.
- **Input:** UTF-8 text string, 10–500 words.
- **Validation:** System shall reject empty inputs, non-English inputs, and inputs with no detectable electrical engineering content.
- **POC:** Single prompt input via CLI or simple text box.
- **Production:** Conversational multi-turn input with context retention.

FR-02: Intent Extraction

- **Description:** System shall extract structured electrical intent from natural language input.
- **Required fields to extract:**
- `voltage_levels` : List of voltage levels mentioned (e.g., [132, 33] kV)
- `topology_type` : Enumerated type (radial, ring, double_bus, single_bus, breaker_and_half, mesh)

- `equipment_list` : Identified equipment with counts (e.g., {transformer: 2, feeder: 6})
- `connectivity_hints` : Explicit connection statements (e.g., "T1 connected to B2")
- `special_arrangements` : Bus coupler, bus section, tie, transfer bus
- **Confidence:** Each extracted field shall include a confidence score (0.0–1.0).
- **Ambiguity:** Fields with confidence < 0.7 shall be flagged for human confirmation.

FR-03: Assumption Generation

- **Description:** When the prompt is underspecified, the system shall generate reasonable engineering defaults and present them to the user.
- **Examples of assumptions:**
- If feeder count unspecified for a substation: assume 4 feeders
- If transformer rating unspecified: assume standard rating for voltage level
- If breaker arrangement unspecified: assume standard breaker-isolator chain
- If bus numbering unspecified: auto-assign from configurable starting number
- **Requirement:** ALL assumptions must be logged and presented to the user before execution.

FR-04: Topology Graph Construction

- **Description:** System shall construct an internal graph G(V, E) representing the electrical topology.
- **Graph properties:**
- Directed or undirected edges based on equipment type
- Nodes typed by equipment category (bus, breaker, transformer, etc.)
- Edges annotated with connection type (electrical, switching, transformer coupling)
- Graph must be connected (no isolated subgraphs unless explicitly specified)
- **Output:** NetworkX graph object + serialized JSON IR.

FR-05: Topology Validation

- **Description:** System shall validate the constructed topology against electrical engineering rules before PSSE execution.
- **Validation checks (minimum):**
- No orphan buses (buses with zero connections)
- No floating transformers (transformer with missing terminal bus)
- No voltage level mismatches at transformer terminals
- No duplicate bus numbers
- Graph connectivity (no unintentional islands)
- Breaker/isolator chains terminate at buses
- At least one source (generator or grid connection) exists
- **Output:** Validation report (pass/fail per check, warnings, errors).

FR-06: Layout Computation

- **Description:** System shall compute (x, y) coordinates for every component in the topology for SLD placement.
- **Constraints:**
- No component overlap (minimum 50px clearance in diagram space)
- Busbars oriented horizontally
- Feeders oriented vertically
- Transformers positioned between voltage levels
- Connection routing uses orthogonal paths
- Maximum 2 line crossings for networks ≤ 20 buses

- **Output:** Coordinate map: `{component_id: (x, y, orientation)}`.

FR-07: PSSE Object Creation

- **Description:** System shall create all required PSSE objects using the PSSE36 Python API (psspy).
- **Objects to create:**
- Buses (bus_data_4)
- Branches/lines (branch_data_3)
- Two-winding transformers (two_winding_data_6)
- Loads (load_data_6)
- Generators/machines (machine_data_4)
- Switched shunts (if needed)
- **Requirement:** Every API call must be logged with parameters and return code.

FR-08: SLD Diagram Placement

- **Description:** System shall place components on the PSSE36 single-line diagram at computed coordinates.
- **Strategy hierarchy:**
- Use PSSE diagram API (if available and documented)
- Use PSSE Python scripting for diagram manipulation
- Fall back to GUI automation (pywinauto) as last resort
- **Requirement:** Diagram must be viewable in PSSE without manual adjustment for simple topologies.

FR-09: File Persistence

- **Description:** System shall save the completed model to disk.
- **Required outputs:**
- `.sav` file (PSSE saved case)
- `.raw` file (raw data export)
- Topology IR (`.json`)
- Execution log (`.json` or `.log`)
- Validation report (`.json`)
- **Requirement:** `.sav` file must load in PSSE36 without errors.

FR-10: Retry on Failure

- **Description:** System shall automatically retry failed operations.
- **Retry policy:**
- Maximum 3 retries per operation
- Maximum 5 retries per workflow
- Exponential backoff: 1s, 2s, 4s between retries
- Different retry strategy per failure type (see §21)
- **Requirement:** After retry budget exhaustion, system shall report failure with full diagnostic log and request human intervention.

FR-11: Human Approval Checkpoint

- **Description:** System shall pause and request human approval before executing PSSE operations.
- **Checkpoint content:**
- Parsed intent summary
- Assumptions made

- Topology graph summary (node count, edge count, voltage levels)
- Validation results
- Estimated execution plan
- **User actions:** Approve, Reject, Modify (with correction text).

FR-12: Execution Logging

- **Description:** Every system action shall be logged in a structured format.
- **Log entry schema:** See §22 for full specification.
- **Requirement:** Logs must be sufficient to reconstruct the entire execution from prompt to final output.

---

§8. Non-Functional Requirements

8.1 Performance

Metric	Target (POC)	Target (Production)
Prompt parsing latency	< 10 seconds	< 5 seconds
Topology construction	< 5 seconds for ≤ 50 buses	< 2 seconds for ≤ 200 buses
Layout computation	< 10 seconds for ≤ 50 buses	< 5 seconds for ≤ 200 buses
PSSE execution (total)	< 60 seconds for ≤ 50 buses	< 30 seconds for ≤ 200 buses
End-to-end (prompt → saved file)	< 3 minutes for ≤ 50 buses	< 1 minute for ≤ 200 buses
LLM API call latency	< 15 seconds per call	< 10 seconds per call

8.2 Reliability

Metric	Target (POC)	Target (Production)
Successful execution rate (simple topologies)	≥ 70%	≥ 95%
Successful execution rate (complex topologies)	≥ 40%	≥ 80%
PSSE file loadable without errors	≥ 90%	≥ 99%
Retry success rate	≥ 30%	≥ 60%
System crash rate	< 10% of runs	< 1% of runs

8.3 Reproducibility

- **Requirement:** Given identical prompt and identical system configuration, the system shall produce identical topology and PSSE model (deterministic output).
- **Implementation:** Fix random seeds, use deterministic graph algorithms, version-pin all dependencies.
- **Exception:** LLM outputs may vary; the system shall cache and reuse parsed intents for reproducibility testing.

8.4 Observability

- **Requirement:** Every execution shall produce a structured log that enables complete reconstruction of the workflow.
- **Log retention:** All logs preserved for the lifetime of the project directory.
- **Log access:** Logs queryable by workflow ID, timestamp range, agent name, action type.

8.5 Portability

- **Platform:** Windows 10/11 x64 with PSSE36 installed.
- **Python:** CPython 3.8–3.11 (must match PSSE36's supported Python version — typically 3.8 or 3.9; verify against installation).
- **No Linux/macOS support required.**
- **No containerization required.**

8.6 Maintainability

- **Code structure:** Modular Python package with clear module boundaries.
- **Documentation:** Every public function shall have a docstring with type annotations.
- **Test coverage:** ≥ 60% line coverage for POC, ≥ 80% for production.
- **Dependency count:** Minimize external dependencies. Target ≤ 15 direct dependencies.

8.7 Security

- **POC:** No security requirements beyond standard file system permissions.
- **Production:** LLM API keys stored in environment variables or local secrets file (not hardcoded). No network exposure. No web server. Local execution only.

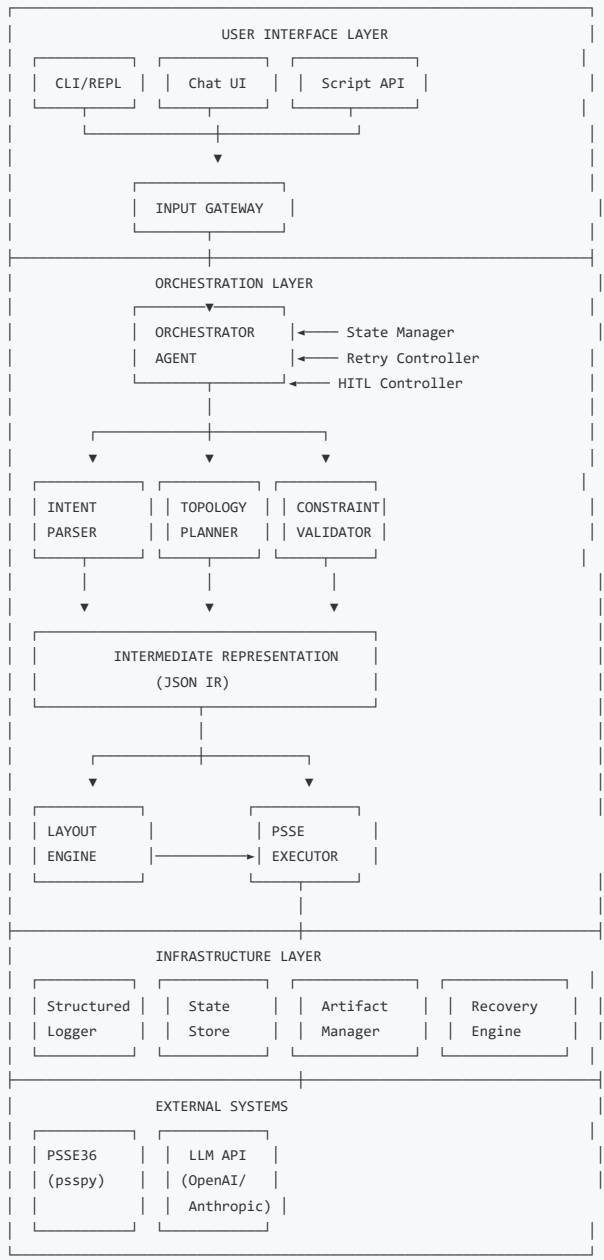
8.8 Usability

- **POC:** CLI interface acceptable. Engineer types prompt, sees structured output, approves/rejects.
- **First-run experience:** Engineer should be able to generate a simple 3-bus network within 5 minutes of installation.
- **Error messages:** All error messages shall include: what failed, why it likely failed, and suggested corrective action.

---

§9. Product Architecture

9.1 High-Level Architecture



9.2 Layer Responsibilities

User Interface Layer

- **Purpose:** Accept user input and present results.
- **POC:** Python CLI using `click` or `typer`. User types prompt, sees structured output.
- **Production:** Desktop GUI (PyQt/Electron) or PSSE plugin panel.

- **Design rule:** UI layer contains ZERO business logic. It is a thin shell that calls the orchestration layer.

Orchestration Layer

- **Purpose:** Coordinate the end-to-end workflow from prompt to artifacts.
- **Contains:** Orchestrator agent, specialist agents, HITL controller, retry controller.
- **Design rule:** All workflow sequencing logic lives here. Specialist agents are stateless functions called by the orchestrator.

Intermediate Representation

- **Purpose:** Canonical data structure representing the electrical topology, independent of both the natural language input and the PSSE data model.
- **Why it exists:** Decouples parsing from execution. Enables validation, layout, and retry without re-parsing. Enables deterministic testing.
- **Design rule:** IR is immutable once validated. Modifications produce a new IR version.

Infrastructure Layer

- **Purpose:** Cross-cutting concerns: logging, state persistence, artifact management, recovery.
- **Design rule:** Infrastructure components are injected as dependencies, not imported directly. This enables testing with mock implementations.

External Systems

- **Purpose:** Third-party systems the application depends on.
- **PSSE36:** Local Windows installation, accessed via psspy Python bindings.
- **LLM API:** Remote API call to frontier language model for intent extraction.
- **Design rule:** All external system calls are wrapped in adapter classes with retry logic, timeout handling, and structured error reporting.

9.3 Architectural Principles

Principle	Description	Rationale
Topology-first	Always construct a graph abstraction before touching PSSE	Enables validation, retry, and debugging without PSSE coupling
IR as contract	The IR is the single source of truth between parsing and execution	Decouples agents, enables deterministic replay
Fail-loud	Every failure is logged with full context, never silently swallowed	Debugging agentic systems requires complete execution traces
Idempotent execution	Re-running the same IR against a clean PSSE session produces identical results	Enables safe retries and reproducibility
Progressive disclosure	Simple prompts produce simple models; complex prompts produce complex models	System complexity scales with input complexity, not system complexity
Graceful degradation	If L3 automation fails, fall back to L2, then L1, then manual	Engineering reliability over automation purity
Determinism over ambiguity	Prefer explicit structured engineering contracts over inference	Electrical topology generation should prefer explicit engineering intent over probabilistic interpretation. Natural language interfaces act as an optional upstream convenience layer, not the primary control plane.

9.4 Data Flow


MVP Entry Point:
Engineering Input (Excel / CSV)
Workbook Loader
Parsed Rows (List[Dict])
Schema Validator
Validated + Type-Coerced Rows
Component Mapper (Deterministic Topology Planning)
TopologyIR (JSON)
Constraint Validator
ValidationReport (JSON)
HITL Checkpoint
ApprovedIR (JSON) or Rejection
Layout Engine
PSSE Executor
Verification
VerificationReport
Artifact Manager
.sav, .raw, IR, logs
Future Entry Point (optional):
User Prompt (text)
Intent Parser (LLM-based)
Structured Schema (same format as Excel rows)
(feeds into Schema Validator above)

9.5 Dependency Order

The pipeline has strict sequential dependencies:

```
Structured Ingestion ⟶ Deterministic Topology Planning ⟶ Validation ⟶ HITL ⟶ Layout ⟶ PSSE Execution ⟶ Verification
```

Why this order matters: 1. **Ingestion → Topology Planning:** Cannot construct a TopologyIR without parsed and validated input data 2. **Topology Planning → Validation:** Cannot validate what hasn't been constructed 3. **Validation → HITL:** Human should review validated (not raw) topology 4. **HITL → Layout:** Don't compute layout for rejected topology 5. **Layout → PSSE:** Need coordinates before placing in PSSE 6. **PSSE → Verification:** Can only verify what has been executed

Parallelism opportunity: Layout computation and PSSE bus creation could theoretically run in parallel (create buses first, then place them). However, for POC, strict sequential execution is simpler and safer.

---

§10. Agentic Architecture

10.1 Architecture Selection: Orchestrator + Specialist Agents

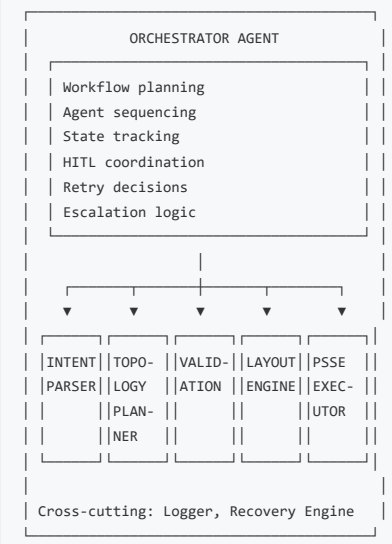
Decision: Use a centralized orchestrator with specialist agents, NOT a multi-agent swarm.

Justification:

Architecture	Pros	Cons	Verdict
Single monolithic agent	Simple, fast to build	Cannot specialize; prompt becomes enormous; hard to debug	Rejected for production
Orchestrator + specialists	Clear responsibilities; debuggable; each agent optimizable independently; manageable complexity	Slightly more code than monolith	**Selected**
Peer-to-peer multi-agent	Flexible; agents negotiate	Unpredictable execution order; hard to debug; coordination overhead; unnecessary for sequential pipeline	Rejected
Hierarchical multi-agent swarm	Handles complex emergent behavior	Massive complexity; non-deterministic; impossible to debug; overkill for this domain	Rejected

The orchestrator + specialist architecture is correct because: 1. The workflow is fundamentally **sequential** (parse → plan → validate → layout → execute) 2. Each stage has **clearly defined inputs and outputs** 3. Each stage requires **different expertise** (NLP, graph theory, geometry, PSSE API) 4. The orchestrator provides a **single point of control** for retry, HITL, and state management 5. Specialist agents are **stateless functions** — easy to test, easy to replace, easy to debug

10.2 Agent Inventory



10.3 Agent Specifications

Agent 1: Orchestrator Agent

Attribute	Specification
Purpose	Decompose the user goal into a sequence of specialist agent invocations, manage workflow state, coordinate HITL checkpoints, and handle retry/escalation decisions.
Inputs	User prompt (text), system configuration, execution history (for retries).
Outputs	Final execution result (success/failure), artifact manifest, execution log.
Uses LLM?	No (POC). Workflow is hardcoded sequential pipeline. Yes (Production) for dynamic replanning on failure.
State	Owns the `ExecutionSession` state machine. Tracks current pipeline stage, retry count, approval status.
Failure modes	Agent sequencing error (wrong order), infinite retry loop, HITL timeout, unhandled exception from specialist.
Recovery	Catch all specialist exceptions. Log full context. Retry with same or modified input up to budget. Escalate to human on budget exhaustion.
POC implementation	Python function `run_pipeline(prompt: str) -> ExecutionResult` with hardcoded sequential stages and try/except around each.
Production implementation	State machine (LangGraph or custom) with conditional routing, dynamic replanning, parallel execution of independent stages.
Complexity	POC: Low (sequential script). Production: Medium (state machine with conditional logic).
Why it must exist	Without a central coordinator, agent sequencing is ad-hoc, retry logic is scattered, and state management is impossible.
Why it should NOT be more complex	The workflow is sequential for POC. A complex orchestrator with dynamic planning is premature until failure patterns are understood from real usage.

Agent 2: Intent Parser Agent

Attribute	Specification
Purpose	Convert natural language prompt into a structured `StructuredIntent` object with extracted electrical parameters.
Inputs	Raw prompt text (str).
Outputs	`StructuredIntent` (Pydantic model) containing voltage_levels, topology_type, equipment_list, connectivity_hints, assumptions, confidence_scores.
Uses LLM?	Yes. This is the primary LLM-dependent component.
Failure modes	LLM hallucination (invents equipment not mentioned), extraction miss (fails to extract stated voltage level), confidence miscalibration, API timeout, malformed JSON response.
Recovery	Retry LLM call with rephrased prompt (up to 2 retries). Validate extracted fields against domain enums. Reject outputs with schema violations. Fall back to asking user for structured input.
POC implementation	Single LLM call with structured output (JSON mode). System prompt includes domain vocabulary, extraction schema, and 5–10 few-shot examples. Response parsed with Pydantic validation.
Production implementation	Two-pass extraction: (1) LLM extracts raw fields, (2) deterministic post-processor validates against domain ontology, applies defaults, resolves ambiguities. Cached prompts for common patterns. Fine-tuned extraction model (if data available).
Complexity	POC: Low-Medium (prompt engineering + Pydantic). Production: Medium (ontology + validation + caching).
Why it must exist	Natural language is the input modality. Something must convert it to structured data.
Why it should NOT be an LLM call for everything	LLMs are non-deterministic and expensive. Once intent is extracted, all downstream processing should be deterministic. The LLM is used once, at the boundary, then discarded.

> *MVP substitution: The Intent Parser Agent is temporarily replaced by a deterministic structured engineering ingestion adapter (Excel / CSV / schema-based input). Future NLP interfaces may later feed into this same structured contract, at which point the Intent Parser Agent resumes its role as specified above.*

Agent 3: Topology Planner Agent

Attribute	Specification
Purpose	Construct an electrical topology graph from the `StructuredIntent`.
Inputs	`StructuredIntent` object.
Outputs	`TopologyGraph` (NetworkX DiGraph) + `TopologyIR` (JSON serialization).
Uses LLM?	No. This is purely deterministic graph construction.
Logic	Pattern-matching on topology_type to select a graph template, then parameterizing with equipment counts and voltage levels.
Example:	`topology_type=double_bus, feeders=6` → generate double-bus template with 2 main buses, bus coupler, 6 feeder bays each containing [isolator → breaker → isolator → CT → feeder_bus].
Failure modes	Unsupported topology type, contradictory equipment specification (e.g., bus coupler in single-bus), missing required parameters.
Recovery	Return validation error to orchestrator. Orchestrator can re-invoke Intent Parser with clarification request.
POC implementation	Python module with template functions: `build_radial()`, `build_single_bus()`, `build_double_bus()`, `build_ring()`. Each function constructs a NetworkX graph procedurally.
Production implementation	Template library stored as parameterized JSON graph templates. Composition engine that can combine templates (e.g., substation + transmission line + substation). Graph algebra for merging subgraphs.
Complexity	POC: Medium (implementing 3–4 topology templates correctly). Production: Medium-High (composition, parameterization, arbitrary topologies).
Why it must exist	The topology graph IS the core abstraction. Without it, the system is just a prompt-to-GUI-click pipeline that cannot be validated, retried, or debugged.

Agent 4: Constraint Validator Agent

Attribute	Specification
Purpose	Validate the constructed topology graph against electrical engineering rules.
Inputs	`TopologyGraph` (NetworkX DiGraph).
Outputs	`ValidationReport` (list of ValidationResult: check_name, severity, passed, message).
Uses LLM?	No. Purely deterministic graph analysis.
Checks implemented (POC, minimum 10):	
1. `no_orphan_buses`	Every bus node has ≥1 edge
2. `no_floating_transformers`	Every transformer has exactly 2 terminal connections
3. `voltage_consistency`	Transformer terminals connect buses of different voltage levels
4. `no_duplicate_ids`	All node IDs are unique
5. `graph_connectivity`	Graph is connected (single component) or isolated components are intentional
6. `source_exists`	At least one generator or grid connection node exists
7. `breaker_chain_valid`	Breakers are between two buses/equipment (not dangling)
8. `feeder_has_endpoint`	Every feeder terminates at a load, line exit, or open end
9. `bus_coupler_valid`	Bus coupler connects exactly 2 buses of same voltage
10. `transformer_winding_count`	Two-winding transformers have exactly 2 terminals
Failure modes	False positive (rejects valid topology), false negative (accepts invalid topology), incomplete check coverage.
Recovery	Return report to orchestrator. Hard failures (severity=ERROR) block execution. Soft failures (severity=WARNING) are logged and presented to user.
POC implementation	Python functions, one per check, operating on NetworkX graph. Return list of `ValidationResult`.
Production implementation	Rule engine with configurable rule sets. Custom rules per utility/client. Machine-learned anomaly detection for novel topologies.
Complexity	POC: Low-Medium. Production: Medium.
Why it must exist	Catching topology errors BEFORE PSSE execution saves minutes of debugging. Invalid topologies will cause PSSE power flow failures that are much harder to diagnose than graph-level checks.

Agent 5: Layout Engine Agent

Attribute	Specification
Purpose	Compute (x, y) coordinates and orientations for all components in the topology graph, producing a readable SLD layout.
Inputs	`TopologyGraph` (NetworkX DiGraph), layout configuration (spacing constants, canvas size).
Outputs	`LayoutMap`: dict mapping component_id → `LayoutPosition(x, y, orientation, layer)`.
Uses LLM?	No. Deterministic geometric computation.
Failure modes	Component overlap, excessive line crossings, layout exceeds canvas bounds, non-aesthetic output.
Recovery	Run collision detection pass. Adjust spacing. Re-run with increased canvas. Score layout and retry with different strategy if score below threshold.
POC implementation	Rule-based heuristic layout. Buses on horizontal rows by voltage level. Equipment placed vertically below parent bus. Fixed spacing grid. See §19 for full specification.
Production implementation	Constraint-based layout solver. Sugiyama-style layered layout adapted for SLD conventions. Layout quality scoring. Multiple layout strategies with best-score selection.
Complexity	POC: Medium (heuristic geometry). Production: High (constraint solver, scoring, multiple strategies).
Why it must exist	SLD without readable layout is useless. PSSE requires explicit coordinates for diagram placement. No auto-layout exists in PSSE.
Why ML/LLM is wrong here	Layout is a geometric constraint problem with deterministic rules. ML would require training data (thousands of labeled SLD layouts) that doesn't exist. Rule-based approaches are more reliable, predictable, and debuggable.

Agent 6: PSSE Executor Agent

Attribute	Specification
Purpose	Translate the validated IR + layout into actual PSSE36 objects and diagram elements by calling the PSSE Python API.
Inputs	`TopologyIR` (JSON), `LayoutMap` (coordinates), PSSE session handle.
Outputs	`ExecutionReport` (list of actions taken, return codes, created object IDs), file paths to saved artifacts.
Uses LLM?	No. Deterministic API calls.
API interactions (psspy):	
- `psspy.bus_data_4(...)`	Create/modify bus
- `psspy.branch_data_3(...)`	Create branch/line
- `psspy.two_winding_data_6(...)`	Create transformer
- `psspy.load_data_6(...)`	Create load
- `psspy.machine_data_4(...)`	Create generator
- `psspy.save(...)`	Save .sav file
- `psspy.rawd_2(...)`	Export .raw file
- Diagram API (TBD)	Place components on SLD
Failure modes	psspy API error (bad parameters), PSSE crash, diagram placement failure, file save failure, session corruption.
Recovery	Log full API call with parameters and return code. On API error: check return code, fix parameters, retry. On PSSE crash: restart PSSE, reinitialize session, replay from IR. On diagram failure: fall back to model-only (no SLD).
POC implementation	Sequential loop over IR components. For each: map IR fields to psspy parameters, call API, check return code, log. Save at end.
Production implementation	Transactional execution with rollback. Batch API calls where possible. Session pooling. Crash detection and auto-recovery.
Complexity	POC: Medium (API mapping + error handling). Production: High (transactions, crash recovery, diagram API).
Why it must exist	This is the only agent that actually creates anything in PSSE. Without it, the system produces plans but no output.

Agent 7: Recovery / Repair Agent

Attribute	Specification
Purpose	Detect failures during execution and attempt automated repair.
Inputs	`FailureReport` from any agent (error type, context, partial state).
Outputs	`RepairAction` (retry, modify_input, skip, escalate) + modified inputs if applicable.
Uses LLM?	POC: No. Production: Optionally, for diagnosing novel failure patterns.
Failure categories:	
Category A: Transient	Network timeout, PSSE busy → simple retry
Category B: Parameter	Invalid API parameters → fix parameters based on error code, retry
Category C: Structural	Invalid topology after execution → re-validate, modify IR, retry
Category D: Fatal	PSSE crash, corrupted session → restart PSSE, replay from IR
Category E: Unrecoverable	Repeated failures after budget → escalate to human
POC implementation	Switch statement on error category. Categories A/B: retry with backoff. Category C: re-run validation. Category D: log and fail. Category E: log and fail.
Production implementation	Repair strategy registry. Pattern matching on error signatures. Automated parameter correction. Partial rollback and resume. LLM-assisted diagnosis for novel failures.
Complexity	POC: Low. Production: Medium-High.
Why it must exist	PSSE API calls fail for numerous reasons. Without automated recovery, every failure requires manual intervention, negating the automation value.
Why it should be simple initially	Most failures in POC will be parameter errors or PSSE quirks. Simple retry with logging handles 80% of cases. Complex repair logic requires failure pattern data that can only be collected through usage.

Agent 8: Audit / Logging Agent (Cross-Cutting)

Attribute	Specification
Purpose	Provide structured logging, execution tracing, and audit trail for every system action.
Inputs	Log events from all other agents (via logging interface).
Outputs	Structured log files (JSON), execution timeline, audit trail.
Uses LLM?	No.
Implementation note	This is NOT a separate agent process. It is a cross-cutting logging infrastructure that all agents use. It is listed as an "agent" for completeness but is implemented as a shared service.
POC implementation	Python `logging` module with structured JSON formatter. Logs written to file per execution session.
Production implementation	SQLite-backed structured log store. Query API for filtering/searching logs. Execution timeline visualization.
Complexity	POC: Low. Production: Medium.
Why it must exist	An agentic system without comprehensive logging is a black box. Every decision, every API call, every failure must be traceable for debugging and trust-building.

10.4 Agent Count Justification

Total agent count: 6 functional agents + 1 orchestrator + 1 cross-cutting service = 8

Why not fewer? - Combining Intent Parser + Topology Planner would mix LLM-dependent code with deterministic graph construction, making testing difficult and creating an unnecessary LLM dependency for graph operations. - Combining Validator + Layout would mix correctness checking with geometry, violating single responsibility. - Removing Recovery means every failure requires human intervention.

Why not more? - No separate "Parameter Resolution Agent" — parameter defaults are handled within Topology Planner. - No separate "Diagram Rendering Agent" — diagram is an output of PSSE Executor. - No separate "Template Selection Agent" — template selection is a function within Topology Planner. - No separate "Prompt Improvement Agent" — prompt refinement is handled by HITL checkpoint. - **Adding agents adds communication overhead, debugging complexity, and code surface area.** Each agent must justify its existence by handling a fundamentally different concern.

10.5 Inter-Agent Communication

Agents communicate through **typed data objects**, not through message passing or event streams.

```
IntentParser.parse(prompt: str) -> StructuredIntent
TopologyPlanner.plan(intent: StructuredIntent) -> TopologyGraph
ConstraintValidator.validate(graph: TopologyGraph) -> ValidationReport
LayoutEngine.compute(graph: TopologyGraph, config: LayoutConfig) -> LayoutMap
PSSEExecutor.execute(ir: TopologyIR, layout: LayoutMap) -> ExecutionReport
RecoveryEngine.repair(failure: FailureReport) -> RepairAction
```

Why typed objects, not messages: - Type checking catches interface mismatches at development time - Pydantic models provide automatic validation - No serialization/deserialization overhead - No message broker to manage - Direct function calls are the simplest possible communication mechanism

Production evolution: If agents ever need to run in separate processes (e.g., PSSE Executor in a monitored subprocess), introduce a thin RPC layer (e.g., ZeroMQ or subprocess pipes). Do NOT build this prematurely.

---

§11. Execution Lifecycle

11.1 Pipeline Stages

```
RECEIVE → PARSE → DISAMBIGUATE → PLAN → VALIDATE → APPROVE → LAYOUT → EXECUTE → VERIFY → PERSIST → RETRY? → COMPLETE
```

11.2 Stage Definitions

Stage 1: RECEIVE

> *MVP Note: For MVP, the system receives a structured engineering input file (Excel / CSV). Future NLP interfaces will receive a text prompt and convert it to the same structured format before entering Stage 2.*

- **Input:** File path to structured engineering workbook (`.xlsx` / `.csv`) — or raw text prompt (future NLP)
- **Output:** Validated file handle or parsed rows, new `workflow_id` assigned
- **State transition:** `IDLE → RECEIVED`
- **Actions:** Validate file path exists, validate file is readable, assign UUID workflow_id, create session directory, log input file path
- **Failure case:** File not found, empty file, or unreadable format → return error immediately
- **Retry:** N/A
- **Logs:** `{event: "input_received", workflow_id, input_type: "excel"|"csv", file_path, row_count, timestamp}`

Stage 2: PARSE (Structured Ingestion)

> *MVP Note: For MVP, parsing is deterministic Excel/CSV ingestion with schema validation. No LLM call is made. Future NLP interfaces will use LLM-based intent extraction in this stage, producing the same structured schema output.*

- **Input:** Parsed rows from workbook loader
- **Output:** `TopologyIR` (Pydantic model) constructed from validated and mapped rows
- **State transition:** `RECEIVED → PARSING`
- **Actions:** Run schema validation (check required columns, types, unique IDs). Coerce numeric fields. Map each row to the appropriate component model (Bus, Branch, Transformer, Load, Generator). Construct TopologyIR.
- **Failure case:** Schema validation errors → collect all errors, report to user, halt. Mapping errors → halt with explicit error message.
- **Retry:** User corrects Excel file and re-submits
- **Logs:** `{event: "schema_validated", row_count, component_counts: {bus: N, branch: N, ...}, validation_errors: [...], timestamp}`

Stage 3: DISAMBIGUATE (Assumption Resolution)

- **Input:** `StructuredIntent` with confidence scores
- **Output:** `ResolvedIntent` with all ambiguities resolved (either by defaults or by user clarification)
- **State transition:** `PARSING → DISAMBIGUATING`
- **Actions:** For each field with confidence < 0.7: apply engineering default OR flag for user question. Generate assumptions list. If critical ambiguity exists (topology_type unclear, voltage missing), pause for user input.
- **Failure case:** User rejects all assumptions → return to RECEIVE with guidance
- **Retry:** User can re-enter modified prompt
- **Logs:** `{event: "assumptions_resolved", assumptions: [{field, default_value, source, confidence}], user_clarifications: [...]}`

Default Assumption Engine (POC):

Missing Field	Default	Rationale
`topology_type` (substation context)	`single_bus`	Simplest arrangement
`feeder_count`	4	Typical small substation
`transformer_count`	1	Minimum for substation
`transformer_rating_mva`	Looked up by voltage level	Standard ratings per kV
`breaker_arrangement`	`breaker_isolator_breaker`	Common arrangement
`bus_numbering_start`	100	Leaves room for future buses
`base_mva`	100	PSSE standard
`load_mw` (per feeder)	10	Placeholder value
`line_impedance`	Standard per voltage/length	Typical conductor data

Stage 4: PLAN (Topology Construction)

- **Input:** `ResolvedIntent`
- **Output:** `TopologyGraph` (NetworkX) + `TopologyIR` (JSON)
- **State transition:** `DISAMBIGUATING → PLANNING`
- **Actions:** Select topology template based on `topology_type`. Instantiate template with parameters from ResolvedIntent. Assign bus numbers, equipment IDs, voltage levels. Construct NetworkX graph. Serialize to JSON IR.
- **Failure case:** Unsupported topology type → error. Contradictory parameters → error with explanation.
- **Retry:** Return to DISAMBIGUATE with specific parameter request
- **Logs:** `{event: "topology_planned", node_count, edge_count, voltage_levels, topology_type, ir_version, ir_hash}`

Stage 5: VALIDATE

- **Input:** `TopologyGraph`
- **Output:** `ValidationReport` (list of check results)
- **State transition:** `PLANNING → VALIDATING`
- **Actions:** Run all validation checks (see §18). Classify results as ERROR (blocking), WARNING (non-blocking), INFO.
- **Failure case:** Any ERROR-severity check fails → return to PLAN with specific failure info
- **Retry:** Orchestrator may re-invoke Topology Planner with constraint adjustments
- **Logs:** `{event: "validation_complete", total_checks, passed, warnings, errors, check_details: [...]}`

Stage 6: APPROVE (Human-in-the-Loop)

- **Input:** `ResolvedIntent`, `TopologyGraph` summary, `ValidationReport`, assumptions list
- **Output:** `ApprovalDecision` (APPROVED | REJECTED | MODIFIED)
- **State transition:** `VALIDATING → AWAITING_APPROVAL`
- **Actions:** Present to user: (a) interpreted intent, (b) assumptions made, (c) topology summary (bus count, equipment list), (d) validation results, (e) estimated actions. Wait for user response.
- **Failure case:** User rejects → return to RECEIVE. User modifies → return to PARSE with modified prompt.
- **Retry:** N/A (user-driven)
- **Logs:** `{event: "approval_checkpoint", presented_summary, user_decision, user_modifications, wait_duration_ms}`

Stage 7: LAYOUT

- **Input:** Approved `TopologyGraph`, `LayoutConfig`
- **Output:** `LayoutMap` (component_id → coordinates)
- **State transition:** `APPROVED → LAYING_OUT`
- **Actions:** Assign vertical layers by voltage level. Position busbars horizontally. Place equipment bays vertically. Compute spacing. Run collision detection. Adjust if needed.
- **Failure case:** Layout exceeds canvas → increase canvas, retry. Unresolvable overlap → log warning, proceed with best-effort.
- **Retry:** Up to 3 layout attempts with increasing spacing
- **Logs:** `{event: "layout_computed", component_count, canvas_size, overlaps_detected, crossings, layout_score}`

Stage 8: EXECUTE (PSSE Creation)

- **Input:** `TopologyIR`, `LayoutMap`, PSSE session
- **Output:** `ExecutionReport` (actions taken, return codes)
- **State transition:** `LAYING_OUT → EXECUTING`
- **Actions:** Initialize PSSE case. Create buses. Create branches. Create transformers. Create loads/generators. Place on diagram. Each API call logged individually.
- **Failure case:** API error → Recovery Agent. PSSE crash → restart and replay. Diagram API unavailable → skip SLD, save model only.
- **Retry:** Per-operation retry (3 attempts). Full replay on session crash.
- **Logs:** `{event: "psse_action", action_type, api_function, parameters, return_code, duration_ms, success}`

Stage 9: VERIFY

- **Input:** PSSE session state after execution
- **Output:** `VerificationReport`
- **State transition:** `EXECUTING → VERIFYING`
- **Actions:** Count created buses (should match IR). Check bus connectivity. Attempt power flow solution (Newton-Raphson). Verify file saves successfully. Check diagram has all components placed.
- **Failure case:** Bus count mismatch → re-execute missing buses. Power flow diverges → log warning (may be expected for incomplete network). Save failure → retry save.
- **Retry:** Verification failures feed back to Recovery Agent
- **Logs:** `{event: "verification_complete", buses_expected, buses_found, pf_converged, file_saved, diagram_complete}`

Stage 10: PERSIST (Artifact Generation)

- **Input:** PSSE session, execution logs, IR, layout
- **Output:** Saved files on disk
- **State transition:** `VERIFYING → PERSISTING`
- **Actions:** Save `.sav` file. Export `.raw` file. Write IR JSON. Write execution log. Write validation report. Write layout metadata. Generate summary report.
- **Failure case:** Disk write error → retry. Permission error → log and report.
- **Retry:** Up to 2 attempts per file
- **Logs:** `{event: "artifact_saved", file_type, file_path, file_size_bytes, checksum}`

Stage 11: RETRY (Conditional)

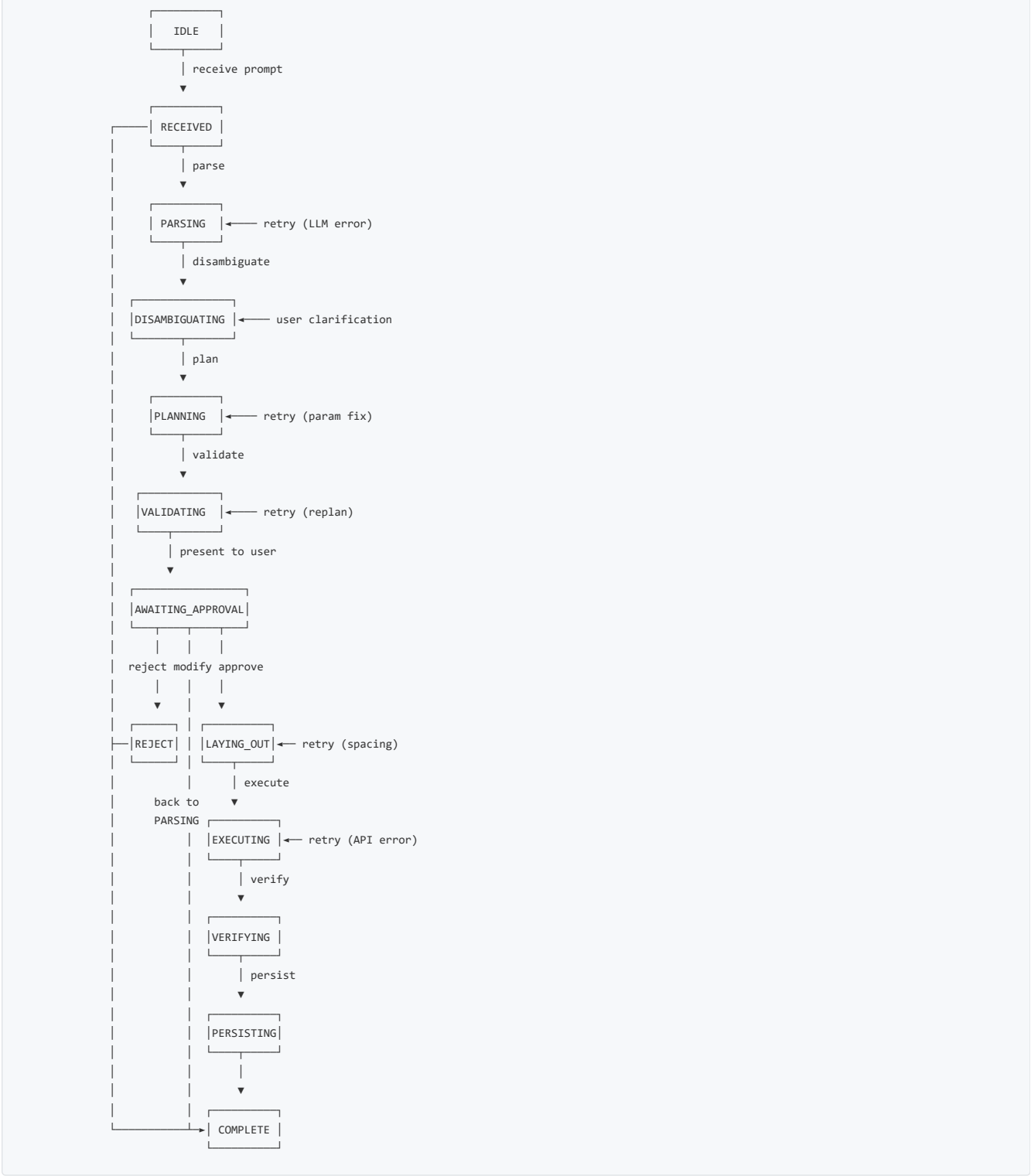
- **Input:** Failure from any previous stage
- **Output:** Retry decision + modified inputs
- **State transition:** `ANY_FAILED → RETRYING → [target stage]`
- **Actions:** Evaluate failure type. Check retry budget. Select repair strategy. Modify inputs if needed. Route back to appropriate stage.

- **Failure case:** Budget exhausted → escalate to human
- **Logs:** `{event: "retry_initiated", original_stage, failure_type, retry_count, repair_strategy, modified_inputs}`

Stage 12: COMPLETE

- **Input:** Successful persistence + verification
- **Output:** Final execution summary
- **State transition:** `PERSISTING → COMPLETE`
- **Actions:** Generate execution summary. Display results to user. Report timing. Log completion.
- **Logs:** `{event: "workflow_complete", workflow_id, total_duration_ms, stages_executed, retries_used, artifacts_generated}`

11.3 State Transition Diagram



§12. Natural Language Parsing

> *MVP Note: Structured engineering input (Excel / CSV / equivalent schemas) serves as the canonical deterministic input mechanism for MVP execution. The natural language parsing system described in this section remains a future convenience layer that maps into the same structured engineering contract and TopologyIR pipeline.*

12.1 Domain Language Characteristics

Power systems engineering language is **semi-structured**, **domain-constrained**, and **surprisingly regular**. Key properties:

1. **Limited vocabulary:** ~200–500 domain terms cover 95% of substation descriptions
2. **Numeric patterns:** Voltage levels (11kV, 33kV, 132kV, 220kV, 400kV), counts, ratings follow predictable formats
3. **Topology keywords:** "double bus", "ring bus", "radial", "breaker-and-half", "bus coupler", "bus section"
4. **Equipment nouns:** "transformer", "breaker", "isolator", "feeder", "CT", "PT", "capacitor", "reactor"
5. **Relationship verbs:** "connected to", "fed from", "tapped off", "coupled with"
6. **Structural patterns:** "X with Y arrangement and Z feeders" is a common template

This regularity means structured extraction is highly feasible with prompt engineering alone — no fine-tuning needed for POC.

12.2 Structured Intent Schema

```
from pydantic import BaseModel, Field
from typing import List, Optional, Dict
from enum import Enum

class TopologyType(str, Enum):
    RADIAL = "radial"
    SINGLE_BUS = "single_bus"
    DOUBLE_BUS = "double_bus"
    RING_BUS = "ring_bus"
    BREAKER_AND_HALF = "breaker_and_half"
    MESH = "mesh"
    SIMPLE_NETWORK = "simple_network"

class EquipmentSpec(BaseModel):
    equipment_type: str          # "transformer", "breaker", "feeder", etc.
    count: int = 1
    rating_mva: Optional[float] = None
    voltage_kv: Optional[float] = None
    additional_params: Dict[str, str] = {}

class ConnectivityHint(BaseModel):
    from_entity: str             # "T1", "Bus1", "feeder_3"
    to_entity: str               # "B2", "main_bus", "grid"
    connection_type: str = "electrical"  # "electrical", "transformer", "switch"

class StructuredIntent(BaseModel):
    voltage_levels_kv: List[float]
    topology_type: TopologyType
    equipment_list: List[EquipmentSpec]
    connectivity_hints: List[ConnectivityHint] = []
    special_arrangements: List[str] = []     # ["bus_coupler", "bus_section"]
    has_ct_pt: bool = False
    has_metering: bool = False
    bus_count_override: Optional[int] = None
    description_summary: str                 # LLM-generated 1-line summary
    confidence_scores: Dict[str, float] = {} # field_name -> 0.0-1.0
    assumptions_made: List[str] = []         # ["Assumed 4 feeders", ...]
    raw_prompt: str
```

12.3 LLM Prompt Design

```
SYSTEM PROMPT:
You are an electrical power systems engineering assistant. Your ONLY task
is to extract structured parameters from natural language descriptions of
electrical substations and networks.

You must output valid JSON matching this exact schema:
{schema_definition}

RULES:
1. Extract ONLY what is explicitly stated or directly implied.
2. For each field, provide a confidence score (0.0 = guessing, 1.0 = certain).
3. If voltage is mentioned as "33kV" extract 33.0. If "132/33kV" extract [132.0, 33.0].
4. If topology type is not explicit, infer from keywords:
   - "double bus" → double_bus
   - "ring" → ring_bus
   - "radial" → radial
   - "breaker and a half" → breaker_and_half
5. Count equipment explicitly mentioned. If "6 feeders" → count=6.
6. If equipment is mentioned without count, assume count=1.
7. List all assumptions in assumptions_made.
8. Do NOT invent equipment not mentioned or implied.

FEW-SHOT EXAMPLES:

Input: "Create a 33kV substation with double bus arrangement and 6 outgoing feeders"
Output: {
  "voltage_levels_kv": [33.0],
  "topology_type": "double_bus",
  "equipment_list": [
    {"equipment_type": "feeder", "count": 6, "voltage_kv": 33.0}
  ],
  "special_arrangements": [],
  "has_ct_pt": false,
  "description_summary": "33kV double bus substation with 6 feeders",
  "confidence_scores": {"voltage_levels_kv": 1.0, "topology_type": 1.0, "equipment_list": 0.9},
  "assumptions_made": ["No transformers specified", "No CT/PT specified", "No bus coupler specified"]
}

Input: "Generate a solar evacuation substation connected to a 132kV grid"
Output: {
  "voltage_levels_kv": [132.0, 33.0],
  "topology_type": "single_bus",
  "equipment_list": [
    {"equipment_type": "transformer", "count": 1, "voltage_kv": 132.0},
    {"equipment_type": "generator", "count": 1, "additional_params": {"type": "solar"}},
    {"equipment_type": "feeder", "count": 1, "voltage_kv": 132.0}
  ],
  "connectivity_hints": [
    {"from_entity": "solar_generator", "to_entity": "grid_132kv", "connection_type": "transformer"}
  ],
  "description_summary": "Solar evacuation substation with 132kV grid connection",
  "confidence_scores": {"voltage_levels_kv": 0.8, "topology_type": 0.6},
  "assumptions_made": ["Assumed 33kV solar side voltage", "Assumed single bus", "Assumed single transformer"]
}

[3 more examples omitted for brevity — include in implementation]

USER INPUT: {user_prompt}
```

12.4 Ambiguity Classification

Ambiguity Level	Example	System Behavior
Unambiguous	"Create 5 buses connected in radial topology at 33kV"	Proceed directly
Low ambiguity	"Create a 33kV double bus substation with 6 feeders"	Apply defaults, log assumptions, proceed
Medium ambiguity	"Create a substation with transformers"	Apply defaults, present assumptions for approval
High ambiguity	"Create a substation"	Request clarification: voltage? topology? equipment?
Invalid	"What is the weather?"	Reject: not an electrical engineering command

12.5 Parser Failure Modes and Mitigations

Failure	Detection	Mitigation
LLM hallucinates equipment	Equipment type not in domain vocabulary enum	Reject unknown types, re-prompt
LLM invents connectivity	Referenced entities don't exist in equipment list	Strip invalid references, warn
LLM returns invalid JSON	JSON parse failure	Retry with stricter format instructions
LLM misidentifies topology	Confidence score < 0.5 on topology_type	Flag for user confirmation
LLM misses stated voltage	Voltage in prompt but not in output	Regex pre-scan for voltage patterns, cross-check
LLM timeout	No response in 30s	Retry up to 2 times, then fail

12.6 Deterministic Pre-Processing (Before LLM)

Before calling the LLM, apply regex-based pre-extraction:

```
import re

def pre_extract(prompt: str) -> dict:
    """Extract high-confidence fields deterministically before LLM."""
    hints = {}

    # Voltage extraction: "33kV", "132 kV", "33/11kV"
    voltages = re.findall(r'(\d+(?:\.\d+)?)\s*k[Vv]', prompt)
    if voltages:
        hints['voltage_levels_kv'] = [float(v) for v in voltages]

    # Equipment counts: "6 feeders", "2 transformers"
    counts = re.findall(r'(\d+)\s+(transformer|feeder|breaker|isolator|bus(?:es)?)', prompt, re.I)
    if counts:
        hints['equipment_counts'] = {eq.lower(): int(n) for n, eq in counts}

    # Topology keywords
    topo_map = {
        'double bus': 'double_bus', 'single bus': 'single_bus',
        'ring bus': 'ring_bus', 'radial': 'radial',
        'breaker and a half': 'breaker_and_half', 'mesh': 'mesh'
    }
    for keyword, topo_type in topo_map.items():
        if keyword in prompt.lower():
            hints['topology_type'] = topo_type
            break

    return hints
```

These deterministic hints serve as **validation anchors**: if the LLM output contradicts a regex-extracted voltage level, the regex value takes precedence (higher confidence for explicit numeric patterns).

---

§13. Human-in-the-Loop (HITL) Architecture

13.1 Why HITL Is Non-Negotiable

Electrical power system models are used for planning, protection, and operational decisions. An incorrect model can lead to: - Mis-sized equipment (financial loss) - Incorrect protection settings (safety risk) - Invalid planning decisions (regulatory risk) - Failed power flow studies (wasted engineering time)

The system must NEVER silently produce an incorrect model. Every assumption, every inferred parameter, every topology decision must be visible to the engineer before execution.

13.2 Checkpoint Design



13.3 Approval Modes

Mode	When Used	Behavior
Strict (default)	All POC usage, critical models	System pauses at every checkpoint. No execution without explicit approval.
Relaxed	Senior engineer, non-critical models	System pauses only for WARNING/ERROR validations. Auto-approves clean topologies.
Batch	Research/testing use cases	System logs all checkpoints but executes without pausing. Post-hoc review.
Skip	Development/testing only	No checkpoints. Dangerous. Requires explicit flag `--no-hitl`.

13.4 Modification Handling

When user selects MODIFY, they can:

1. **Text correction:** Type a correction string (e.g., "Actually use 8 feeders, not 6") → System re-runs Intent Parser with original prompt + correction appended
2. **Parameter override:** Directly edit the StructuredIntent JSON (advanced users) → System re-runs from Topology Planning
3. **Reject and rephrase:** Type entirely new prompt → System starts from RECEIVE

13.5 Trust Calibration Over Time

```
POC Phase:     100% checkpoints → every execution reviewed
Early Usage:   90% checkpoints → skip for known-good templates
Mature Usage:  50% checkpoints → auto-approve simple topologies
Production:    Configurable per user/topology complexity
```

Implementation: Track `approval_history` per topology template. If a template has been approved 10+ times without modification, suggest auto-approve for that template in relaxed mode.

13.6 POC vs Production HITL

Aspect	POC	Production
Checkpoint display	CLI text output	Rich UI panel with graph visualization
Modification input	Free text correction	Structured form + direct IR editing
Approval persistence	JSON log file	Database with audit trail
Rollback capability	Re-run from scratch	Undo to any checkpoint state
Multi-user approval	N/A	Approval workflows with roles

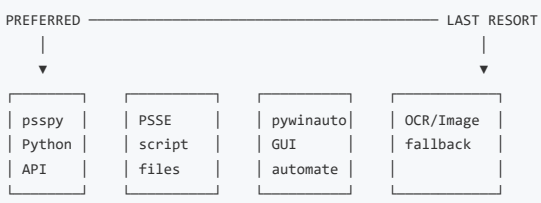
---

§14. PSSE36 Integration Strategy

14.1 PSSE36 Technical Profile

- **Vendor:** Siemens PTI (formerly Shaw/PTI)
- **Platform:** Windows only (x64)
- **Version:** 36 (targeting latest point release)
- **Python bridge:** `psspy` module — Python API binding to PSSE internals
- **Python version:** Typically CPython 3.8 or 3.9 (verify against PSSE36 installation)
- **API style:** Procedural C-style functions exposed to Python
- **Session model:** Single PSSE instance per process. Global state. Not thread-safe.
- **File formats:** `.sav` (binary saved case), `.raw` (text-based raw data), `.sld` (diagram)
- **Diagram system:** Proprietary SLD editor with its own coordinate system

14.2 Integration Strategy Hierarchy



Strategy 1: Native psspy API (Primary — MUST USE)

What it covers: - Bus creation: `psspy.bus_data_4(ibus, intgar, realar, name)` - Branch creation: `psspy.branch_data_3(ibus, jbus, ckt, intgar, realar)` - Transformer creation: `psspy.two_winding_data_6(ibus, jbus, ckt, intgar, realar, ...)` - Load creation: `psspy.load_data_6(ibus, loadid, intgar, realar)` - Machine creation: `psspy.machine_data_4(ibus, macid, intgar, realar)` - Save/load: `psspy.save(filepath)`, `psspy.case(filepath)`, `psspy.read(ibus, filepath)` - Raw export: `psspy.rawd_2(sid, filnam, ...)` - Power flow: `psspy.fnsl(...)`, `psspy.fdns(...)`

What it likely does NOT cover (requires investigation): - Diagram object creation (placing buses on SLD canvas) - Diagram coordinate assignment - Diagram connection routing - Diagram element styling

POC implementation:

```
import psspy

def initialize_psse():
    """Initialize PSSE and start a new case."""
    psspy.psseinit(buses=500)  # Initialize with max bus capacity
    psspy.newcase_2(
        sbase=100.0,          # System base MVA
        title1="Auto-generated model",
        title2=f"Created {datetime.now().isoformat()}"
    )

def create_bus(bus_num: int, name: str, base_kv: float, bus_type: int = 1):
    """Create a bus in the PSSE case.
    bus_type: 1=load bus, 2=generator bus, 3=swing bus
    """
    ierr = psspy.bus_data_4(
        ibus=bus_num,
        intgar=[bus_type, 1, 0, 0],  # [TYPE, AREA, ZONE, OWNER]
        realar=[base_kv, 0.0, 0.0, 1.0, 0.0, 0.0, 1.1, 0.9],
        name=name
    )
    if ierr != 0:
        raise PSSEError(f"bus_data_4 failed for bus {bus_num}: error code {ierr}")
    return ierr

def create_branch(from_bus: int, to_bus: int, ckt: str, r_pu: float, x_pu: float, b_pu: float):
    """Create a branch (line) between two buses."""
    ierr = psspy.branch_data_3(
        ibus=from_bus,
        jbus=to_bus,
        ckt=ckt,
        intgar=[1, 1, 0, 0, 0, 0],
        realar=[r_pu, x_pu, b_pu, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
    )
    if ierr != 0:
        raise PSSEError(f"branch_data_3 failed {from_bus}->{to_bus}: error code {ierr}")
    return ierr

def create_transformer(hv_bus: int, lv_bus: int, ckt: str, r_pu: float, x_pu: float,
                       sbase_mva: float, windv1: float = 1.0, windv2: float = 1.0):
    """Create a two-winding transformer."""
    ierr = psspy.two_winding_data_6(
        ibus=hv_bus,
        jbus=lv_bus,
        ckt=ckt,
        intgar=[...],  # Control modes, tap settings
        realar=[r_pu, x_pu, sbase_mva, windv1, windv2, ...],
        # Detailed parameter mapping required — see PSSE API docs
    )
    if ierr != 0:
        raise PSSEError(f"two_winding_data_6 failed {hv_bus}->{lv_bus}: error code {ierr}")
    return ierr
```

Strategy 2: PSSE Script Files (Secondary)

If specific API calls are problematic, generate PSSE-compatible `.idv` (IDEV) script files:

```
BUS DATA
    100, 'BUS100      ', 33.0000, 1, 1, 1, 1, 1.00000, 0.0000
    101, 'BUS101      ', 33.0000, 1, 1, 1, 1, 1.00000, 0.0000
END OF BUS DATA
BRANCH DATA
    100, 101, '1 ', 1, 1, 0.01000, 0.10000, 0.02000
END OF BRANCH DATA
```

Then load via `psspy.read(0, filepath)` or `psspy.readrawversion(filepath)`.

When to use: If direct API calls have undocumented parameter issues or if batch creation is more reliable than sequential calls.

Strategy 3: GUI Automation via pywinauto (Fallback)

```
from pywinauto import Application

def gui_place_bus_on_diagram(app, bus_num, x, y):
    """Last resort: automate PSSE GUI to place bus on diagram."""
    main_window = app.window(title_re=".*PSSE.*")
    # Navigate to diagram editor
    main_window.menu_select("Diagram->Open")
    # Use keyboard/mouse automation to place component
    # THIS IS FRAGILE AND VERSION-DEPENDENT
```

When to use: ONLY for diagram placement if no API exists. Document all GUI interaction sequences. Version-lock to specific PSSE36 build.

Strategy 4: OCR/Image-Based (Emergency Only)

Using screenshot analysis to verify diagram state. Not for creation, only for verification.

When to use: Post-execution verification that diagram looks correct. Compare screenshot against expected layout.

14.3 Diagram API Investigation Plan

The diagram/SLD API is the **highest technical risk** in this project. The following investigation must happen in P0:

```
Investigation Checklist:
□ 1. Import psspy, list all functions matching "diagram", "diag", "sld", "draw", "place"
       >>> [f for f in dir(psspy) if any(k in f.lower() for k in ['diag','sld','draw','place'])]
□ 2. Check for pssdiag module (separate diagram API module)
□ 3. Check PSSE36 installation directory for diagram-related Python files
□ 4. Check PSSE API documentation PDF for diagram chapter
□ 5. Search for "slider" or "diagram" in PSSE example scripts
□ 6. Test: create a case with buses, manually add to diagram, save .sld, inspect file format
□ 7. Test: can .sld file be generated/edited programmatically (text? XML? binary?)
□ 8. Check if PSSE has a macro recorder that captures diagram actions as Python
□ 9. Search PSSE user forums and documentation for programmatic diagram creation
□ 10. Contact Siemens PTI support if API documentation is insufficient
```

14.4 Session Management

```
class PSSESessionManager:
    """Manages PSSE36 session lifecycle."""

    def __init__(self):
        self.is_initialized = False
        self.case_path = None
        self.bus_count = 0

    def initialize(self, max_buses: int = 500):
        """Initialize PSSE. Must be called once per process."""
        if not self.is_initialized:
            import psspy
            psspy.psseinit(buses=max_buses)
            self.is_initialized = True

    def new_case(self, sbase: float = 100.0, title: str = ""):
        """Start a fresh case. Idempotent — clears any existing case."""
        psspy.newcase_2(sbase=sbase, title1=title)
        self.bus_count = 0
        self.case_path = None

    def save(self, filepath: str):
        """Save current case to .sav file."""
        ierr = psspy.save(filepath)
        if ierr != 0:
            raise PSSEError(f"Save failed: {ierr}")
        self.case_path = filepath

    def export_raw(self, filepath: str):
        """Export current case to .raw file."""
        ierr = psspy.rawd_2(sid=0, filnam=filepath)
        if ierr != 0:
            raise PSSEError(f"Raw export failed: {ierr}")
```

14.5 Idempotent Execution

Requirement: Re-running the executor with the same IR against a clean PSSE session MUST produce identical results.

Implementation: 1. Always start with `psspy.newcase_2()` — never modify existing case 2. Bus numbers are deterministically assigned from IR (not auto-incremented) 3. All parameters come from IR (no runtime-dependent values) 4. Save file path includes workflow_id for uniqueness

14.6 Crash Recovery

```
PSSE Crash Detection:
1. Wrap every psspy call in try/except
2. Check return codes (non-zero = error)
3. Monitor PSSE process via psutil
4. If process disappears → crash detected

Recovery Procedure:
1. Log crash details (last successful action, last attempted action)
2. Wait 5 seconds for PSSE process cleanup
3. Re-import psspy (may require fresh Python process)
4. Re-initialize PSSE session
5. Replay all actions from IR up to the crashed point
6. Retry the failed action with modified parameters if applicable
7. If repeated crash → abort and report to user

POC Implementation:
- Catch exceptions → log → fail with diagnostic
- No automated restart in POC (too complex, low ROI until failure patterns understood)

Production Implementation:
- Subprocess isolation: run PSSE executor in child process
- Parent process monitors child health
- On crash: respawn child, replay from checkpoint
```

14.7 File Persistence Strategy

File	Format	Generation Method	Purpose
`{workflow_id}.sav`	Binary (PSSE)	`psspy.save()`	Loadable PSSE case
`{workflow_id}.raw`	Text (PSSE)	`psspy.rawd_2()`	Portable raw data
`{workflow_id}_topology.json`	JSON	IR serialization	Topology record
`{workflow_id}_layout.json`	JSON	Layout serialization	Coordinate record
`{workflow_id}_execution.json`	JSON	Structured logger	Full execution log
`{workflow_id}_validation.json`	JSON	Validator output	Validation report
`{workflow_id}.sld`	PSSE diagram	PSSE diagram save	Single-line diagram

14.8 Known PSSE API Quirks (From Experience)

1. **psspy.psseinit() must be called exactly once** — calling it again can corrupt state
2. **Bus numbers must be positive integers** — no zero, no negative
3. **Branch circuit IDs are strings** — typically "1", "2", etc.
4. **Return code 0 = success**, any other value = error (but error code meanings vary by function)
5. **String parameters have fixed width** — bus names are typically 12 characters, padded
6. **psspy functions are NOT thread-safe** — never call from multiple threads
7. **PSSE output goes to its own console** — redirect with `psspy.progress_output()` and `psspy.alert_output()`
8. **Import order matters** — `import psspy` before any PSSE operations, and ensure PSSE's Python path is in sys.path

---

§15. Internal Representation (IR)

15.1 Why IR Is Critical

The IR is the **single most important data structure** in the system. It serves as:

1. **Contract between agents:** Intent Parser produces it, Topology Planner builds it, Validator checks it, Layout Engine reads it, PSSE Executor consumes it.
2. **Debugging artifact:** When something goes wrong, the IR shows exactly what was planned.
3. **Replay mechanism:** Given the same IR, the executor produces identical PSSE output.
4. **Retry enabler:** On failure, the system retries from the IR, not from the original prompt.
5. **Version history:** IR versions track how the topology evolved through modifications.

Without IR, the system is a black box. Prompt goes in, PSSE model comes out, and when it fails nobody can determine why.

15.2 IR JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "PSSE36 Topology IR",
  "type": "object",
  "required": ["metadata", "buses", "connections"],
  "properties": {
    "metadata": {
      "type": "object",
      "properties": {
        "ir_version": {"type": "string", "pattern": "^\\d+\\.\\d+\\.\\d+$"},
        "workflow_id": {"type": "string", "format": "uuid"},
        "created_at": {"type": "string", "format": "date-time"},
        "source_prompt": {"type": "string"},
        "topology_type": {"type": "string", "enum": ["radial","single_bus","double_bus","ring_bus","breaker_and_half","mesh","simple_network"]},
        "base_mva": {"type": "number", "default": 100.0},
        "assumptions": {"type": "array", "items": {"type": "string"}},
        "description": {"type": "string"}
      }
    },
    "buses": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["bus_id", "bus_number", "name", "base_kv", "bus_type"],
        "properties": {
          "bus_id": {"type": "string"},
          "bus_number": {"type": "integer", "minimum": 1},
          "name": {"type": "string", "maxLength": 12},
          "base_kv": {"type": "number", "minimum": 0.1},
          "bus_type": {"type": "integer", "enum": [1, 2, 3, 4]},
          "area": {"type": "integer", "default": 1},
          "zone": {"type": "integer", "default": 1},
          "owner": {"type": "integer", "default": 1},
          "vm_pu": {"type": "number", "default": 1.0},
          "va_deg": {"type": "number", "default": 0.0},
          "vmax_pu": {"type": "number", "default": 1.1},
          "vmin_pu": {"type": "number", "default": 0.9},
          "category": {"type": "string", "enum": ["main_bus","auxiliary_bus","feeder_bus","transfer_bus","internal_bus"]},
          "voltage_level_kv": {"type": "number"},
          "diagram_hints": {
            "type": "object",
            "properties": {
              "layer": {"type": "integer"},
              "position_hint": {"type": "string", "enum": ["top","middle","bottom"]},
              "orientation": {"type": "string", "enum": ["horizontal","vertical"]}
            }
          }
        }
      }
    },
    "branches": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["branch_id", "from_bus", "to_bus", "branch_type"],
        "properties": {
          "branch_id": {"type": "string"},
          "from_bus": {"type": "integer"},
          "to_bus": {"type": "integer"},
          "circuit_id": {"type": "string", "default": "1"},
          "branch_type": {"type": "string", "enum": ["line","cable","breaker","isolator","disconnect","bus_coupler","bus_section","zero_impedance"]},
          "r_pu": {"type": "number", "default": 0.0},
          "x_pu": {"type": "number", "default": 0.0001},
          "b_pu": {"type": "number", "default": 0.0},
          "rate_a_mva": {"type": "number", "default": 0.0},
          "rate_b_mva": {"type": "number", "default": 0.0},
          "rate_c_mva": {"type": "number", "default": 0.0},
          "length_km": {"type": "number"},
          "status": {"type": "integer", "enum": [0, 1], "default": 1}
```

```
        }
      }
    },
    "transformers": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["transformer_id", "hv_bus", "lv_bus"],
        "properties": {
          "transformer_id": {"type": "string"},
          "hv_bus": {"type": "integer"},
          "lv_bus": {"type": "integer"},
          "circuit_id": {"type": "string", "default": "1"},
          "name": {"type": "string"},
          "r_pu": {"type": "number", "default": 0.005},
          "x_pu": {"type": "number", "default": 0.10},
          "sbase_mva": {"type": "number"},
          "windv1_pu": {"type": "number", "default": 1.0},
          "windv2_pu": {"type": "number", "default": 1.0},
          "tap_min": {"type": "number"},
          "tap_max": {"type": "number"},
          "tap_step": {"type": "number"},
          "vector_group": {"type": "string"},
          "status": {"type": "integer", "enum": [0, 1], "default": 1}
        }
      }
    },
    "loads": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["load_id", "bus_number"],
        "properties": {
          "load_id": {"type": "string"},
          "bus_number": {"type": "integer"},
          "load_identifier": {"type": "string", "default": "1"},
          "p_mw": {"type": "number", "default": 10.0},
          "q_mvar": {"type": "number", "default": 5.0},
          "status": {"type": "integer", "enum": [0, 1], "default": 1}
        }
      }
    },
    "generators": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["gen_id", "bus_number"],
        "properties": {
          "gen_id": {"type": "string"},
          "bus_number": {"type": "integer"},
          "machine_id": {"type": "string", "default": "1"},
          "p_mw": {"type": "number"},
          "q_mvar": {"type": "number"},
          "qmax_mvar": {"type": "number"},
          "qmin_mvar": {"type": "number"},
          "pmax_mw": {"type": "number"},
          "pmin_mw": {"type": "number"},
          "gen_type": {"type": "string", "enum": ["thermal","hydro","solar","wind","grid"]},
          "status": {"type": "integer", "enum": [0, 1], "default": 1}
        }
      }
    },
    "connections": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["from_id", "to_id", "connection_type"],
        "properties": {
          "from_id": {"type": "string"},
          "to_id": {"type": "string"},
          "connection_type": {"type": "string", "enum": ["electrical","switching","transformer_coupling","bus_tie"]}
        }
```

```
      }
    }
  }
}
```

15.3 Example IR: Simple Radial Network (5 Buses)

```
{
  "metadata": {
    "ir_version": "1.0.0",
    "workflow_id": "a1b2c3d4-...",
    "created_at": "2026-05-22T12:00:00Z",
    "source_prompt": "Create a radial network with 5 buses at 33kV",
    "topology_type": "radial",
    "base_mva": 100.0,
    "assumptions": ["Bus 1 is swing bus", "Default line impedance used", "10MW load on each non-source bus"],
    "description": "Simple 5-bus radial network at 33kV"
  },
  "buses": [
    {"bus_id": "B1", "bus_number": 101, "name": "BUS-101", "base_kv": 33.0, "bus_type": 3, "category": "main_bus"},
    {"bus_id": "B2", "bus_number": 102, "name": "BUS-102", "base_kv": 33.0, "bus_type": 1, "category": "main_bus"},
    {"bus_id": "B3", "bus_number": 103, "name": "BUS-103", "base_kv": 33.0, "bus_type": 1, "category": "main_bus"},
    {"bus_id": "B4", "bus_number": 104, "name": "BUS-104", "base_kv": 33.0, "bus_type": 1, "category": "main_bus"},
    {"bus_id": "B5", "bus_number": 105, "name": "BUS-105", "base_kv": 33.0, "bus_type": 1, "category": "main_bus"}
  ],
  "branches": [
    {"branch_id": "L1", "from_bus": 101, "to_bus": 102, "branch_type": "line", "r_pu": 0.01, "x_pu": 0.10, "b_pu": 0.02},
    {"branch_id": "L2", "from_bus": 102, "to_bus": 103, "branch_type": "line", "r_pu": 0.01, "x_pu": 0.10, "b_pu": 0.02},
    {"branch_id": "L3", "from_bus": 103, "to_bus": 104, "branch_type": "line", "r_pu": 0.01, "x_pu": 0.10, "b_pu": 0.02},
    {"branch_id": "L4", "from_bus": 104, "to_bus": 105, "branch_type": "line", "r_pu": 0.01, "x_pu": 0.10, "b_pu": 0.02}
  ],
  "transformers": [],
  "loads": [
    {"load_id": "LD2", "bus_number": 102, "p_mw": 10.0, "q_mvar": 5.0},
    {"load_id": "LD3", "bus_number": 103, "p_mw": 10.0, "q_mvar": 5.0},
    {"load_id": "LD4", "bus_number": 104, "p_mw": 10.0, "q_mvar": 5.0},
    {"load_id": "LD5", "bus_number": 105, "p_mw": 10.0, "q_mvar": 5.0}
  ],
  "generators": [
    {"gen_id": "G1", "bus_number": 101, "p_mw": 50.0, "q_mvar": 0.0, "gen_type": "grid"}
  ],
  "connections": [
    {"from_id": "B1", "to_id": "B2", "connection_type": "electrical"},
    {"from_id": "B2", "to_id": "B3", "connection_type": "electrical"},
    {"from_id": "B3", "to_id": "B4", "connection_type": "electrical"},
    {"from_id": "B4", "to_id": "B5", "connection_type": "electrical"}
  ]
}
```

15.4 Example IR: 33kV Double Bus Substation (Abbreviated)


```
{
  "metadata": {
    "topology_type": "double_bus",
    "source_prompt": "33kV double bus substation with 4 feeders and bus coupler",
    "assumptions": ["Standard breaker-isolator arrangement", "Bus coupler between main buses"]
  },
  "buses": [
    {"bus_id": "MB1", "bus_number": 100, "name": "MAIN-BUS-1", "base_kv": 33.0, "bus_type": 3, "category": "main_bus"},
    {"bus_id": "MB2", "bus_number": 101, "name": "MAIN-BUS-2", "base_kv": 33.0, "bus_type": 1, "category": "main_bus"},
    {"bus_id": "BC_B", "bus_number": 102, "name": "COUPLER-BUS", "base_kv": 33.0, "bus_type": 1, "category": "internal_bus"},
    {"bus_id": "F1_B", "bus_number": 110, "name": "FEEDER-1", "base_kv": 33.0, "bus_type": 1, "category": "feeder_bus"},
    {"bus_id": "F2_B", "bus_number": 111, "name": "FEEDER-2", "base_kv": 33.0, "bus_type": 1, "category": "feeder_bus"},
    {"bus_id": "F3_B", "bus_number": 112, "name": "FEEDER-3", "base_kv": 33.0, "bus_type": 1, "category": "feeder_bus"},
    {"bus_id": "F4_B", "bus_number": 113, "name": "FEEDER-4", "base_kv": 33.0, "bus_type": 1, "category": "feeder_bus"}
  ],
  "branches": [
    {"branch_id": "BC", "from_bus": 100, "to_bus": 101, "branch_type": "bus_coupler", "r_pu": 0.0, "x_pu": 0.0001},
    {"branch_id": "F1_CB", "from_bus": 100, "to_bus": 110, "branch_type": "breaker", "r_pu": 0.0, "x_pu": 0.0001},
    {"branch_id": "F2_CB", "from_bus": 100, "to_bus": 111, "branch_type": "breaker", "r_pu": 0.0, "x_pu": 0.0001},
    {"branch_id": "F3_CB", "from_bus": 101, "to_bus": 112, "branch_type": "breaker", "r_pu": 0.0, "x_pu": 0.0001},
    {"branch_id": "F4_CB", "from_bus": 101, "to_bus": 113, "branch_type": "breaker", "r_pu": 0.0, "x_pu": 0.0001}
  ],
  "loads": [
    {"load_id": "LD1", "bus_number": 110, "p_mw": 10.0, "q_mvar": 5.0},
    {"load_id": "LD2", "bus_number": 111, "p_mw": 10.0, "q_mvar": 5.0},
    {"load_id": "LD3", "bus_number": 112, "p_mw": 10.0, "q_mvar": 5.0},
    {"load_id": "LD4", "bus_number": 113, "p_mw": 10.0, "q_mvar": 5.0}
  ]
}
```


15.5 MVP Structured Engineering Input Contract

For the MVP, the system supports a deterministic, row-wise structured engineering schema. This tabular contract represents the canonical source of truth for topology creation, bypassing natural language ambiguity while remaining compatible with optional upstream LLM/NLP translation layers.

Input Formats and Ingestion

- **Supported Formats:** Microsoft Excel (`.xlsx`), Comma-Separated Values (`.csv`), or equivalent structured JSON schemas.
- **Row Semantic:** Each row in the table represents exactly one distinct electrical component.
- **No Positional Dependency:** The system MUST NOT infer network topology or relationships from row ordering. All node connectivity and device associations must be explicitly defined via identifiers in the fields.
- **Mapping to TopologyIR:** Rows map deterministically into the `TopologyIR` using the mapping sequence: `Excel/CSV Row` → `Schema Validation` → `TopologyIR` → `Topology Validation` → `PSSE Execution`

Core Contract Fields

The schema is built around explicit identifiers and topology connections: * `component_type` : The class of the electrical component (e.g., `bus` , `branch` , `transformer` , `load` , `generator`). * `component_id` : A unique alphanumeric string identifying the device. * `from_bus` : Unique integer/string identifying the starting terminal bus connection. * `to_bus` : Unique integer/string identifying the ending terminal bus connection (for branches/transformers).

Concise Example Schema Table

component_type	component_id	from_bus	to_bus	bus_type	base_kv	mva	r_pu	x_pu	p_mw	q_mvar	vector_
bus	BUS101	101		swing	33						
branch	BR1	101	102				0.01	0.1			
transformer	TX1	102	201		33	13.2	0.008	0.0796			YNd11
load	LD1	201			0.66				2.0	0.5	

Note: This schema represents the MVP contract structure. Detailed parameters (such as R, X, P, Q, vector groups) map into corresponding fields in the `TopologyIR` dynamically, keeping the implementation flexible.

---

§16. Graph Topology Engine

16.1 Core Abstraction

The electrical network is represented as a graph G(V, E) where: - **V (nodes):** Every distinct electrical point — buses, equipment terminals, junction points - **E (edges):** Every electrical connection — lines, cables, switching devices, transformer windings

16.2 Node Types

Node Type	PSSE Mapping	Properties
`main_bus`	Bus record (type 1/2/3)	base_kv, area, zone
`auxiliary_bus`	Bus record (type 1)	Used for internal switching
`feeder_bus`	Bus record (type 1)	Terminal of feeder exit
`transfer_bus`	Bus record (type 1)	Transfer bus in double-bus
`generator_terminal`	Machine on bus	p_mw, q_mvar, gen_type
`load_terminal`	Load on bus	p_mw, q_mvar

16.3 Edge Types

Edge Type	PSSE Mapping	Impedance
`line`	Branch	R + jX, charging B
`cable`	Branch	R + jX, high charging B
`breaker`	Branch (zero-impedance)	~0 + j0.0001
`isolator`	Branch (zero-impedance)	~0 + j0.0001
`bus_coupler`	Branch (zero-impedance)	~0 + j0.0001
`transformer`	Two-winding transformer	R + jX on system base

16.4 Template-Based Construction

```
import networkx as nx

def build_double_bus_substation(
    bus_start: int,
    voltage_kv: float,
    feeder_count: int,
    has_bus_coupler: bool = True
) -> nx.DiGraph:
    """Build a double-bus substation topology."""
    G = nx.DiGraph()

    # Main buses
    mb1 = bus_start
    mb2 = bus_start + 1
    G.add_node(mb1, type="main_bus", base_kv=voltage_kv, name=f"MAIN-BUS-1", category="main_bus")
    G.add_node(mb2, type="main_bus", base_kv=voltage_kv, name=f"MAIN-BUS-2", category="main_bus")

    # Bus coupler
    if has_bus_coupler:
        bc = bus_start + 2
        G.add_node(bc, type="auxiliary_bus", base_kv=voltage_kv, name="COUPLER", category="internal_bus")
        G.add_edge(mb1, bc, type="breaker", branch_type="bus_coupler")
        G.add_edge(bc, mb2, type="breaker", branch_type="bus_coupler")

    # Feeder bays - distribute across both buses
    next_bus = bus_start + 10
    for i in range(feeder_count):
        fb = next_bus + i
        parent_bus = mb1 if i < feeder_count // 2 else mb2

        G.add_node(fb, type="feeder_bus", base_kv=voltage_kv, name=f"FEEDER-{i+1}", category="feeder_bus")

        # Breaker connection: main_bus → breaker → feeder_bus
        G.add_edge(parent_bus, fb, type="breaker", branch_type="breaker")

    return G
```

16.5 Graph Operations

Operation	Implementation	Use Case
`compose(G1, G2, tie_points)`	`nx.compose()` + edge addition	Connecting substation to transmission
`validate(G)`	Custom traversal checks	Pre-execution validation
`serialize(G)`	Custom JSON serializer	IR persistence
`deserialize(json)`	Custom JSON loader	IR loading
`layout(G)`	Hierarchical layout algorithm	Coordinate assignment
`diff(G1, G2)`	Node/edge set comparison	Change tracking

---

§17. Topology Validation System

17.1 Validation Check Registry

```
class ValidationSeverity(Enum):
    ERROR = "error"      # Blocks execution
    WARNING = "warning"  # Logged, user notified, execution continues
    INFO = "info"        # Logged only

class ValidationResult:
    check_name: str
    severity: ValidationSeverity
    passed: bool
    message: str
    affected_entities: List[str]
```

17.2 Validation Checks (Complete List)

#	Check Name	Severity	Logic	POC	Prod
1	`no_orphan_buses`	ERROR	Every bus has ≥1 connected edge	✓	✓
2	`no_floating_transformers`	ERROR	Every transformer has both HV and LV bus	✓	✓
3	`voltage_consistency`	ERROR	Transformer HV bus kV > LV bus kV	✓	✓
4	`no_duplicate_bus_numbers`	ERROR	All bus_number values unique	✓	✓
5	`graph_connectivity`	ERROR	Graph has exactly 1 connected component	✓	✓
6	`source_exists`	ERROR	≥1 generator or swing bus (type 3)	✓	✓
7	`valid_bus_types`	ERROR	Bus types ∈	✓	✓
8	`positive_impedance`	WARNING	All branches have X > 0 (except zero-impedance)	✓	✓
9	`breaker_termination`	WARNING	Breakers connect between two buses	✓	✓
10	`feeder_has_endpoint`	WARNING	Feeder buses have load or line exit	✓	✓
11	`bus_coupler_same_voltage`	ERROR	Bus coupler connects same-kV buses	✓	✓
12	`no_self_loops`	ERROR	No branch from bus X to bus X	✓	✓
13	`bus_number_range`	WARNING	Bus numbers in 1–999999 range	✓	✓
14	`unique_branch_ids`	ERROR	(from_bus, to_bus, ckt) tuples are unique	✓	✓
15	`reasonable_impedance`	WARNING	Branch impedance within expected range for voltage		✓
16	`transformer_rating_check`	WARNING	Transformer MVA rating reasonable for voltage		✓
17	`load_balance_check`	INFO	Total generation ≈ total load (within 20%)		✓

17.3 Validation Execution

```
def validate_topology(graph: nx.DiGraph, ir: TopologyIR) -> ValidationReport:
    checks = [
        check_no_orphan_buses,
        check_no_floating_transformers,
        check_voltage_consistency,
        check_no_duplicate_bus_numbers,
        check_graph_connectivity,
        check_source_exists,
        check_valid_bus_types,
        check_positive_impedance,
        check_breaker_termination,
        check_feeder_endpoint,
        check_bus_coupler_voltage,
        check_no_self_loops,
        check_bus_number_range,
        check_unique_branch_ids,
    ]

    results = []
    for check_fn in checks:
        result = check_fn(graph, ir)
        results.append(result)

    has_errors = any(r.severity == "error" and not r.passed for r in results)

    return ValidationReport(
        results=results,
        passed=not has_errors,
        error_count=sum(1 for r in results if r.severity == "error" and not r.passed),
        warning_count=sum(1 for r in results if r.severity == "warning" and not r.passed)
    )
```

17.4 Auto-Fix Capabilities (Production)

Issue	Auto-Fix	Strategy
Orphan bus	Add to nearest bus via zero-impedance branch	Closest bus by bus_number proximity
Missing source	Designate bus 1 as swing bus	Set bus_type = 3
Zero impedance on line	Set to minimum default	x_pu = 0.0001
Duplicate bus number	Renumber with offset	Add 1000 to conflicting bus

Auto-fixes MUST be logged as assumptions and presented in HITL checkpoint.

---

§18. Component Library

18.1 Supported Components

Component: Bus

Attribute	Value
PSSE type	Bus record
IR fields	bus_number, name, base_kv, bus_type, area, zone
Placement rules	Main buses: horizontal, full-width. Auxiliary buses: shorter segments near parent.
Orientation	Horizontal (default), Vertical (for compact layouts)
Visual convention	Thick horizontal line. Label above or below. Color by voltage level.
Validation	Unique number, valid type (1-4), positive base_kv

Component: Breaker (Circuit Breaker)

Attribute	Value
PSSE type	Zero-impedance branch (R≈0, X=0.0001)
IR fields	from_bus, to_bus, branch_type="breaker", status
Placement rules	Between parent bus and equipment. Vertically aligned with bay.
Orientation	Vertical (inline with bay)
Visual convention	Small square or diamond symbol
Validation	Must connect two buses. Status 0 (open) or 1 (closed).

Component: Isolator / Disconnect Switch

Attribute	Value
PSSE type	Zero-impedance branch or status flag on adjacent branch
IR fields	from_bus, to_bus, branch_type="isolator", status
Placement rules	Adjacent to breaker, between breaker and bus
Orientation	Vertical (inline with bay)
Visual convention	Open/closed blade symbol
Validation	Must be adjacent to breaker in equipment chain

Component: Transformer (Two-Winding)

Attribute	Value
PSSE type	Two-winding transformer record
IR fields	hv_bus, lv_bus, r_pu, x_pu, sbase_mva, windv1, windv2, tap settings
Placement rules	Between voltage levels. HV terminal connects to HV bus, LV to LV bus. Vertically between bus layers.
Orientation	Vertical (connecting two horizontal bus layers)
Visual convention	Two overlapping circles
Validation	HV bus kV > LV bus kV. Positive impedance. Rating > 0.

Component: Generator / Machine

Attribute	Value
PSSE type	Machine record on a bus
IR fields	bus_number, machine_id, p_mw, q_mvar, gen_type
Placement rules	Connected to generator bus. Placed above or to the side of bus.
Orientation	Vertical (above bus)
Visual convention	Circle with "G" or "~" symbol
Validation	Connected bus must be type 2 (PV) or 3 (swing).

Component: Load

Attribute	Value
PSSE type	Load record on a bus
IR fields	bus_number, load_identifier, p_mw, q_mvar
Placement rules	Connected to load bus. Placed below bus or at feeder terminal.
Orientation	Vertical (below bus)
Visual convention	Downward arrow or triangle
Validation	Connected to valid bus. P and Q have reasonable values.

Component: Line

Attribute	Value
PSSE type	Branch record
IR fields	from_bus, to_bus, r_pu, x_pu, b_pu, rate_mva, length_km
Placement rules	Orthogonal routing between bus positions
Orientation	Horizontal or vertical depending on bus positions
Visual convention	Single line between buses
Validation	Positive impedance. Connected buses exist.

Component: CT (Current Transformer)

Attribute	Value
PSSE type	Not directly modelled (represented as annotation or auxiliary bus)
IR fields	location (between breaker and bus), associated_bus
Placement rules	Inline with feeder bay, between breaker and feeder exit
Visual convention	Small circle or CT symbol
Validation	Must be in series with a breaker

Component: PT (Potential/Voltage Transformer)

Attribute	Value
PSSE type	Not directly modelled (annotation)
IR fields	location, associated_bus
Placement rules	Connected to bus via tee connection
Visual convention	Small circle with "PT" or parallel lines
Validation	Connected to a bus

Component: Bus Coupler

Attribute	Value
PSSE type	Zero-impedance branch between two main buses
IR fields	from_bus (main bus 1), to_bus (main bus 2), branch_type="bus_coupler"
Placement rules	Vertical between the two horizontal main buses. Centered or at designated position.
Visual convention	Breaker symbol between two busbars
Validation	Connects two buses of same voltage level

Component: Capacitor Bank

Attribute	Value
PSSE type	Switched shunt or fixed shunt on bus
IR fields	bus_number, mvar_rating, steps
Placement rules	Below or beside connected bus
Visual convention	Parallel lines (capacitor symbol)
Validation	Positive MVAR. Connected to valid bus.

Component: Reactor

Attribute	Value
PSSE type	Switched shunt (inductive) or branch
IR fields	bus_number, mvar_rating
Placement rules	Below or beside connected bus
Visual convention	Coil symbol
Validation	Negative MVAR (inductive). Connected to valid bus.

Component: Feeder (Outgoing Line)

Attribute	Value
PSSE type	Branch + terminal bus + load (composite)
IR fields	parent_bus, feeder_bus, breaker, load
Placement rules	Vertical bay dropping from main bus. Contains: isolator→breaker→isolator→CT→feeder exit.
Visual convention	Vertical line with equipment symbols, terminating in arrow or load
Validation	Complete equipment chain. Connected to main bus.

Component: Bus Section Switch

Attribute	Value
PSSE type	Zero-impedance branch within a single bus (splits bus into sections)
IR fields	bus_section_1, bus_section_2, branch_type="bus_section"
Placement rules	Inline with horizontal busbar, dividing it into segments
Visual convention	Breaker symbol inline with busbar
Validation	Both sections same voltage.

Component: Meter / Metering Point

Attribute	Value
PSSE type	Annotation only
IR fields	location, meter_type
Placement rules	Near CT/PT location
Visual convention	"M" or meter symbol
Validation	Must have associated CT or PT

---

§19. Layout / Placement Engine

19.1 Why This Is the Hardest Problem

SLD layout is a constrained optimization problem combining: 1. **Graph drawing** (node placement, edge routing) 2. **Domain conventions** (busbars horizontal, feeders vertical) 3. **Aesthetic requirements** (readability, symmetry, minimal crossings) 4. **Physical constraints** (no overlap, minimum spacing)

Force-directed layouts (e.g., spring embeddings) are **wrong for SLDs** because: - They produce organic/circular layouts, not the rigid orthogonal layouts engineers expect - They ignore domain conventions (busbars should be straight horizontal lines) - They don't respect hierarchy (voltage levels should form distinct layers) - They are non-deterministic (different runs produce different layouts)

19.2 Layout Algorithm: Hierarchical Layer-Based

```
ALGORITHM: HierarchicalSLDLayout

INPUT: TopologyGraph G, LayoutConfig C
OUTPUT: LayoutMap (component_id → (x, y, orientation))

Step 1: LAYER ASSIGNMENT
  - Assign each bus to a vertical layer based on voltage level
  - Highest voltage = top layer (y = 0)
  - Each lower voltage level = next layer down
  - Within same voltage: main buses on primary layer, auxiliary buses on sub-layer

Step 2: HORIZONTAL ORDERING
  - Within each layer, order buses by:
    a. Main buses first, then auxiliary
    b. Feeders ordered left-to-right by bus_number
    c. Bus coupler centered between main buses
    d. Transformers aligned with their connecting buses

Step 3: COORDINATE ASSIGNMENT
  - Bus y-coordinate = layer_index * LAYER_SPACING
  - Bus x-coordinate = position_in_layer * BUS_SPACING
  - Equipment x-coordinate = parent_bus.x (vertically aligned)
  - Equipment y-coordinate = parent_bus.y + offset_by_type

Step 4: BAY LAYOUT
  For each feeder bay:
    - Start at parent bus position
    - Stack downward: isolator (y+1), breaker (y+2), isolator (y+3), CT (y+4), feeder exit (y+5)
    - Each element at same x as parent bus connection point

Step 5: COLLISION DETECTION
  For each pair of components:
    - If bounding boxes overlap: shift right component by MIN_SPACING
    - If connection lines cross: try swapping horizontal positions

Step 6: EDGE ROUTING
  For each connection:
    - Use orthogonal routing (horizontal + vertical segments only)
    - Route around obstacles
    - Minimize total wire length
    - Minimize crossings

Step 7: FINAL ADJUSTMENT
  - Center the layout on canvas
  - Apply grid snapping (round to nearest GRID_SIZE)
  - Verify all components within canvas bounds
```

19.3 Layout Constants (POC Defaults)

```
class LayoutConfig:
    # Spacing (in PSSE diagram units — investigate actual units)
    LAYER_SPACING = 200        # Vertical distance between voltage levels
    BUS_SPACING = 100          # Horizontal distance between parallel buses
    BAY_WIDTH = 80             # Width of a feeder bay
    EQUIPMENT_SPACING = 40     # Vertical spacing between equipment in a bay
    BUSBAR_LENGTH = None       # Auto-calculated: feeder_count * BAY_WIDTH + margins
    MIN_CLEARANCE = 30         # Minimum clearance between any two components
    GRID_SIZE = 10             # Snap-to-grid size

    # Canvas
    CANVAS_WIDTH = 2000
    CANVAS_HEIGHT = 1500
    MARGIN = 100

    # Orientation
    BUSBAR_ORIENTATION = "horizontal"
    FEEDER_ORIENTATION = "vertical"
    TRANSFORMER_ORIENTATION = "vertical"
```

19.4 Layout Modes

Mode	Description	Algorithm Variant
Radial	Linear chain of buses	Single horizontal row, buses left to right
Single Bus	One busbar with feeders	Busbar at top, feeders dropping vertically
Double Bus	Two parallel busbars with feeders	Two horizontal busbars, feeders between or below, coupler centered
Ring Bus	Circular bus arrangement	Circular or octagonal node placement
Breaker-and-Half	Three breakers per bay between two buses	Specialized bay layout with shared breakers
Transmission	Point-to-point lines between substations	Wide horizontal layout, substations as clusters

19.5 Layout Scoring Function

```
def score_layout(layout: LayoutMap, graph: TopologyGraph) -> float:
    """Score layout quality. Higher = better. Range 0.0 to 1.0."""

    overlap_penalty = count_overlaps(layout) * -0.2
    crossing_penalty = count_line_crossings(layout, graph) * -0.1
    spacing_score = evaluate_spacing_uniformity(layout) * 0.3
    symmetry_score = evaluate_symmetry(layout) * 0.2
    alignment_score = evaluate_alignment(layout) * 0.2
    bounds_score = 1.0 if all_within_canvas(layout) else 0.0

    total = (
        bounds_score * 0.1 +
        max(0, 1.0 + overlap_penalty) * 0.3 +
        max(0, 1.0 + crossing_penalty) * 0.2 +
        spacing_score +
        symmetry_score +
        alignment_score
    )
    return min(1.0, max(0.0, total))
```

19.6 POC vs Production Layout

Aspect	POC	Production
Algorithm	Fixed heuristic rules	Multiple strategies, best score wins
Collision resolution	Shift-right	Constraint solver (OR-tools)
Edge routing	Direct orthogonal	A* pathfinding around obstacles
Scoring	Not used	Score-based strategy selection
Templates	Hardcoded coordinates	Parameterized template library
Canvas adaptation	Fixed size	Dynamic sizing based on content

---

§20. Diagram Generation Strategy

20.1 Diagram Pipeline



20.2 PSSE Diagram API Investigation Results (To Be Determined)

The exact PSSE36 diagram API capabilities must be determined experimentally during P0. Possible scenarios:

Scenario A: Full Diagram API Exists - Functions like `psspy.diagram_add_bus(bus_num, x, y)` exist - Connections are drawn automatically or via `psspy.diagram_add_line(...)` - **Best case.** Direct programmatic SLD creation.

Scenario B: Partial Diagram API - Can create diagram objects but not position them freely - Or can position but not route connections - **Moderate case.** Hybrid API + manual adjustment.

Scenario C: No Diagram API - Diagram manipulation is GUI-only - Must use pywinauto or similar for placement - **Worst case.** GUI automation required.

Scenario D: Diagram File is Editable - `.sld` file is text/XML-based and can be edited directly - Generate file programmatically without PSSE GUI - **Viable alternative.** Bypass PSSE entirely for diagram.

20.3 Fallback Strategy: Model-First, Diagram-Second

If diagram API proves intractable:

1. **Phase 1:** Create PSSE model (buses, branches, transformers) — this is well-supported by psspy
2. **Phase 2:** Save .sav and .raw files — model is complete and usable
3. **Phase 3:** Generate diagram separately — either via discovered API, GUI automation, or as standalone SVG/PNG export from the layout engine

The model is the primary deliverable. The diagram is a visual aid. A valid PSSE model without a diagram is still valuable. A diagram without a valid model is useless.

20.4 Alternative Diagram Output (If PSSE Diagram Proves Impossible)

Generate an independent SLD visualization using: - **SVG export:** Render the layout as an SVG file using Python (svgwrite library) - **HTML/Canvas:** Interactive browser-based SLD viewer - **Matplotlib:** Static PNG/PDF diagram

This is NOT a substitute for the PSSE native diagram but provides immediate visual feedback while the PSSE diagram integration is being developed.

```
import svgwrite

def export_sld_svg(layout: LayoutMap, graph: TopologyGraph, filepath: str):
    """Generate standalone SVG diagram from layout."""
    dwg = svgwrite.Drawing(filepath, size=("2000px", "1500px"))

    for component_id, pos in layout.items():
        node_data = graph.nodes[component_id]
        if node_data["type"] == "main_bus":
            # Draw horizontal busbar
            dwg.add(dwg.line(
                start=(pos.x - 200, pos.y),
                end=(pos.x + 200, pos.y),
                stroke="black", stroke_width=3
            ))
            dwg.add(dwg.text(node_data["name"], insert=(pos.x, pos.y - 10), font_size=12))
        elif node_data["type"] == "breaker":
            # Draw breaker symbol
            dwg.add(dwg.rect(
                insert=(pos.x - 5, pos.y - 5),
                size=(10, 10), fill="black"
            ))
        # ... more component types

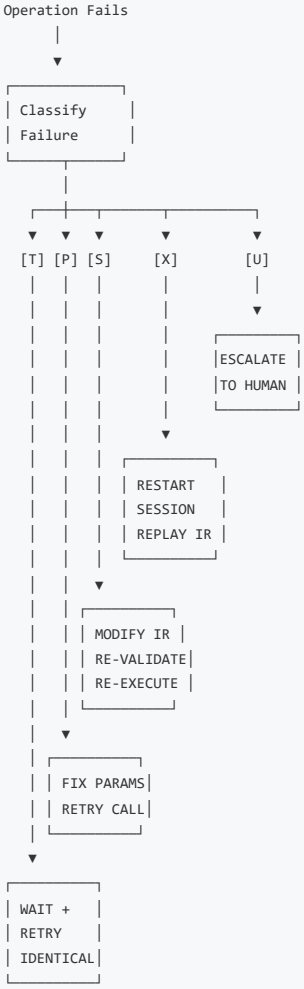
    dwg.save()
```

§21. Recovery + Retry Engine

21.1 Failure Taxonomy

Category	Code	Examples	Retry Strategy	Max Retries
Transient	T	LLM timeout, network glitch, PSSE busy	Wait + retry identical call	3
Parameter	P	Invalid bus number, wrong API args, type mismatch	Fix parameter, retry	2
Structural	S	Invalid topology post-execution, missing connection	Re-validate, modify IR, re-execute	1
Session	X	PSSE crash, corrupted state, process death	Restart session, replay from IR	1
Unrecoverable	U	Repeated failures, fundamental API incompatibility	Escalate to human	0

21.2 Retry Flow



21.3 Retry Budget

```
class RetryBudget:
    max_retries_per_operation: int = 3
    max_retries_per_workflow: int = 8
    max_session_restarts: int = 1
    backoff_base_seconds: float = 1.0
    backoff_multiplier: float = 2.0
    backoff_max_seconds: float = 30.0

    def get_delay(self, attempt: int) -> float:
        delay = self.backoff_base_seconds * (self.backoff_multiplier ** attempt)
        return min(delay, self.backoff_max_seconds)
```

21.4 Escalation Logic

```
IF retry_count >= max_retries_per_operation:
    IF failure_category in [T, P]:
        Try alternative approach (e.g., script file instead of API)
    ELIF failure_category == S:
        Present IR + failure to human for manual correction
    ELIF failure_category == X:
        IF session_restarts < max_session_restarts:
            Restart and replay
        ELSE:
            Escalate to human

IF total_workflow_retries >= max_retries_per_workflow:
    ABORT workflow
    Generate diagnostic report
    Present to human
```

21.5 Partial Execution Recovery

Problem: PSSE execution creates objects sequentially. If it fails at bus #15 out of 20, buses 1–14 already exist.

POC strategy: Start fresh. `newcase_2()` wipes everything. Replay entire IR from scratch. Simple, safe, idempotent.

Production strategy: Checkpoint after each major phase (all buses created, all branches created, all transformers created). On failure, roll back to last checkpoint and retry from there.

```
class ExecutionCheckpoint:
    checkpoint_id: str
    phase: str            # "buses_created", "branches_created", "transformers_created"
    created_objects: List[str]  # IDs of successfully created objects
    timestamp: datetime
    psse_state_hash: str  # Hash of current PSSE state for verification
```

---

§22. Logging + Observability

22.1 Structured Log Schema

Every log entry follows this schema:

```
class LogEntry:
    timestamp: datetime          # ISO 8601
    workflow_id: str            # UUID for the entire workflow
    step_id: str                # UUID for this specific step
    sequence_number: int        # Monotonically increasing per workflow
    agent_name: str             # "intent_parser", "topology_planner", etc.
    action_type: str            # "llm_call", "graph_construction", "psse_api", etc.
    severity: str               # "DEBUG", "INFO", "WARNING", "ERROR", "CRITICAL"
    component_type: Optional[str]  # "bus", "transformer", "breaker", etc.
    component_id: Optional[str]    # Specific component identifier
    input_summary: str          # Truncated input description
    output_summary: str         # Truncated output description
    duration_ms: Optional[float]
    success: bool
    error_message: Optional[str]
    error_code: Optional[str]
    recovery_action: Optional[str]
    metadata: Dict[str, Any]    # Agent-specific additional data
```

22.2 Log Categories

Category	Agent	Example Entries
`PROMPT`	Orchestrator	Raw prompt received, sanitized prompt
`LLM`	Intent Parser	API call sent, response received, tokens used
`INTENT`	Intent Parser	Extracted fields, confidence scores, assumptions
`TOPOLOGY`	Topology Planner	Graph constructed, node/edge counts, template used
`VALIDATION`	Validator	Each check result, pass/fail, affected entities
`HITL`	Orchestrator	Checkpoint presented, user decision, modifications
`LAYOUT`	Layout Engine	Coordinates assigned, collisions detected, score
`PSSE`	PSSE Executor	Each API call, parameters, return code, timing
`DIAGRAM`	PSSE Executor	Diagram placement actions, coordinates used
`FILE`	Artifact Manager	Files saved, paths, sizes, checksums
`RETRY`	Recovery Engine	Failure classified, retry strategy, attempt number
`STATE`	State Manager	State transitions, checkpoint created/restored

22.3 Verbosity Modes

Mode	Levels Captured	Use Case
`MINIMAL`	ERROR, CRITICAL	Production, minimal disk usage
`STANDARD`	INFO, WARNING, ERROR, CRITICAL	Normal operation
`VERBOSE`	DEBUG + all above	Debugging, development
`TRACE`	All + full input/output payloads	Deep debugging, replay

POC default: `VERBOSE`
Production default: `STANDARD` with `TRACE` available on-demand per workflow

22.4 Log Storage

POC: JSON Lines file per workflow session.

```
project/logs/
  {workflow_id}/
    execution.jsonl      # All log entries, one JSON per line
    summary.json         # High-level execution summary
    timeline.json        # Ordered event timeline for visualization
```

Production evolution: SQLite database with indexed columns for workflow_id, agent_name, severity, timestamp. Query API for filtering and searching.

```
CREATE TABLE execution_logs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    workflow_id TEXT NOT NULL,
    step_id TEXT NOT NULL,
    sequence_number INTEGER NOT NULL,
    timestamp TEXT NOT NULL,
    agent_name TEXT NOT NULL,
    action_type TEXT NOT NULL,
    severity TEXT NOT NULL,
    component_type TEXT,
    component_id TEXT,
    input_summary TEXT,
    output_summary TEXT,
    duration_ms REAL,
    success BOOLEAN NOT NULL,
    error_message TEXT,
    metadata_json TEXT,
    UNIQUE(workflow_id, sequence_number)
);
CREATE INDEX idx_workflow ON execution_logs(workflow_id);
CREATE INDEX idx_severity ON execution_logs(severity);
CREATE INDEX idx_agent ON execution_logs(agent_name);
```

22.5 Replayability

Goal: Given a workflow_id, reconstruct the exact execution sequence.

Implementation: 1. Log entries are ordered by `sequence_number` within a workflow 2. Each entry captures sufficient input/output to understand the action 3. The IR is versioned — each modification creates a new version 4. PSSE API calls are logged with full parameters 5. Given the logs, a developer can manually replay each step

Production: Automated replay — feed logged IR into executor, compare outputs.

22.6 Timing Metrics

Captured automatically for every operation:

```
@contextmanager
def timed_operation(logger, agent_name, action_type, **kwargs):
    start = time.perf_counter()
    try:
        yield
        duration = (time.perf_counter() - start) * 1000
        logger.log(agent_name=agent_name, action_type=action_type,
                   duration_ms=duration, success=True, **kwargs)
    except Exception as e:
        duration = (time.perf_counter() - start) * 1000
        logger.log(agent_name=agent_name, action_type=action_type,
                   duration_ms=duration, success=False, error_message=str(e), **kwargs)
        raise
```

---

§23. State Management

23.1 Execution State Machine

```
class WorkflowState(Enum):
    IDLE = "idle"
    RECEIVED = "received"
    PARSING = "parsing"
    DISAMBIGUATING = "disambiguating"
    PLANNING = "planning"
    VALIDATING = "validating"
    AWAITING_APPROVAL = "awaiting_approval"
    LAYING_OUT = "laying_out"
    EXECUTING = "executing"
    VERIFYING = "verifying"
    PERSISTING = "persisting"
    RETRYING = "retrying"
    COMPLETED = "completed"
    FAILED = "failed"
    REJECTED = "rejected"

VALID_TRANSITIONS = {
    "idle": ["received"],
    "received": ["parsing", "failed"],
    "parsing": ["disambiguating", "retrying", "failed"],
    "disambiguating": ["planning", "parsing", "failed"],
    "planning": ["validating", "retrying", "failed"],
    "validating": ["awaiting_approval", "retrying", "failed"],
    "awaiting_approval": ["laying_out", "rejected", "parsing"],
    "laying_out": ["executing", "retrying", "failed"],
    "executing": ["verifying", "retrying", "failed"],
    "verifying": ["persisting", "retrying", "failed"],
    "persisting": ["completed", "failed"],
    "retrying": ["parsing", "planning", "validating", "laying_out", "executing", "failed"],
    "completed": [],
    "failed": [],
    "rejected": [],
}
```

23.2 Session State Object

```
class ExecutionSession:
    workflow_id: str
    state: WorkflowState
    created_at: datetime
    updated_at: datetime

    # Input
    raw_prompt: str

    # Intermediate products
    structured_intent: Optional[StructuredIntent]
    resolved_intent: Optional[ResolvedIntent]
    topology_graph: Optional[nx.DiGraph]
    topology_ir: Optional[TopologyIR]
    validation_report: Optional[ValidationReport]
    approval_decision: Optional[ApprovalDecision]
    layout_map: Optional[LayoutMap]
    execution_report: Optional[ExecutionReport]
    verification_report: Optional[VerificationReport]

    # Retry tracking
    retry_count: int = 0
    retry_history: List[RetryRecord] = []

    # Artifacts
    artifact_paths: Dict[str, str] = {}  # artifact_type -> file_path

    # Timing
    stage_timings: Dict[str, float] = {}  # stage_name -> duration_ms
```

23.3 State Persistence

POC: In-memory `ExecutionSession` object. Serialized to JSON on completion or failure. Lost on crash.

Production: SQLite-backed. Session state written to DB after every state transition. Enables crash recovery: on restart, load incomplete sessions and resume from last completed state.

23.4 Idempotency Guarantees

Stage	Idempotent?	Strategy
Parse	Yes (with cached LLM response)	Cache response by prompt hash
Plan	Yes	Deterministic template + parameters
Validate	Yes	Deterministic checks on deterministic graph
Layout	Yes	Deterministic algorithm with fixed seed
Execute	Yes (with clean session)	Always start from newcase_2()
Verify	Yes	Read-only checks
Persist	Yes	Overwrite existing files with same workflow_id

§24. Data Model

24.1 Entity Definitions

```
# === Core Entities ===

class BusEntity:
    bus_id: str              # Internal unique ID ("B1", "MB1", "F3_B")
    bus_number: int          # PSSE bus number (100, 101, ...)
    name: str                # PSSE bus name (max 12 chars)
    base_kv: float           # Base voltage in kV
    bus_type: int            # 1=PQ, 2=PV, 3=swing, 4=isolated
    category: BusCategory    # main_bus, auxiliary, feeder, transfer, internal
    area: int = 1
    zone: int = 1
    owner: int = 1
    vm_pu: float = 1.0
    va_deg: float = 0.0
    lifecycle: EntityLifecycle = EntityLifecycle.PLANNED
    # PLANNED → VALIDATED → CREATED_IN_PSSE → PLACED_ON_DIAGRAM → VERIFIED

class TransformerEntity:
    transformer_id: str
    name: str
    hv_bus: int              # HV terminal bus number
    lv_bus: int              # LV terminal bus number
    circuit_id: str = "1"
    r_pu: float = 0.005
    x_pu: float = 0.10
    sbase_mva: float = 100.0
    windv1_pu: float = 1.0
    windv2_pu: float = 1.0
    vector_group: str = "YNd11"
    lifecycle: EntityLifecycle = EntityLifecycle.PLANNED

class BranchEntity:
    branch_id: str
    from_bus: int
    to_bus: int
    circuit_id: str = "1"
    branch_type: BranchType  # line, cable, breaker, isolator, bus_coupler, etc.
    r_pu: float = 0.01
    x_pu: float = 0.10
    b_pu: float = 0.02
    rate_a_mva: float = 0.0
    status: int = 1          # 1=in-service, 0=out-of-service
    lifecycle: EntityLifecycle = EntityLifecycle.PLANNED

class LoadEntity:
    load_id: str
    bus_number: int
    load_identifier: str = "1"
    p_mw: float = 10.0
    q_mvar: float = 5.0
    status: int = 1
    lifecycle: EntityLifecycle = EntityLifecycle.PLANNED

class GeneratorEntity:
    gen_id: str
    bus_number: int
    machine_id: str = "1"
    p_mw: float = 50.0
    q_mvar: float = 0.0
    gen_type: GenType        # thermal, hydro, solar, wind, grid
    status: int = 1
    lifecycle: EntityLifecycle = EntityLifecycle.PLANNED

# === Layout Entities ===
```

```
class LayoutPosition:
    component_id: str
    x: float
    y: float
    orientation: Orientation  # HORIZONTAL, VERTICAL
    layer: int                # Voltage level layer
    bay_index: Optional[int]  # Position within bay (for bay equipment)

# === Execution Entities ===

class PSSEAction:
    action_id: str
    action_type: str          # "create_bus", "create_branch", etc.
    api_function: str         # "psspy.bus_data_4", etc.
    parameters: Dict
    return_code: int
    success: bool
    timestamp: datetime
    duration_ms: float

class ValidationResult:
    check_name: str
    severity: ValidationSeverity
    passed: bool
    message: str
    affected_entities: List[str]

class RetryRecord:
    attempt_number: int
    failed_stage: str
    failure_category: str
    failure_message: str
    repair_action: str
    timestamp: datetime
    result: str               # "success", "failed", "escalated"

# === Enums ===

class EntityLifecycle(Enum):
    PLANNED = "planned"
    VALIDATED = "validated"
    CREATED_IN_PSSE = "created_in_psse"
    PLACED_ON_DIAGRAM = "placed_on_diagram"
    VERIFIED = "verified"
    FAILED = "failed"

class BusCategory(Enum):
    MAIN_BUS = "main_bus"
    AUXILIARY_BUS = "auxiliary_bus"
    FEEDER_BUS = "feeder_bus"
    TRANSFER_BUS = "transfer_bus"
    INTERNAL_BUS = "internal_bus"

class BranchType(Enum):
    LINE = "line"
    CABLE = "cable"
    BREAKER = "breaker"
    ISOLATOR = "isolator"
    DISCONNECT = "disconnect"
    BUS_COUPLER = "bus_coupler"
    BUS_SECTION = "bus_section"
    ZERO_IMPEDANCE = "zero_impedance"

class Orientation(Enum):
    HORIZONTAL = "horizontal"
    VERTICAL = "vertical"
```

24.2 Naming Conventions

Entity	ID Pattern	PSSE Name Pattern	Example
Main bus	`MB{n}`	`MAIN-BUS-{n}`	MB1, MAIN-BUS-1
Feeder bus	`F{n}_B`	`FEEDER-{n}`	F3_B, FEEDER-3
Auxiliary bus	`AUX{n}`	`AUX-{n}`	AUX1, AUX-1
Transformer	`T{n}`	`XFMR-{n}`	T1, XFMR-1
Branch/line	`L{n}`	N/A	L1
Breaker	`CB{n}`	N/A	CB3
Load	`LD{n}`	N/A	LD2
Generator	`G{n}`	N/A	G1

24.3 Versioning

Every IR mutation creates a new version:

```
v1.0.0 — Initial construction from intent
v1.1.0 — After validation auto-fixes
v1.2.0 — After human modifications
v2.0.0 — After structural retry (re-planned)
```

Version history is preserved in the execution log.

---

§25. Artifact Management

25.1 Generated Artifacts

Artifact	Format	Generator	Required	Description
PSSE saved case	`.sav`	PSSE Executor	Yes	Binary PSSE case file, loadable in PSSE36
PSSE raw data	`.raw`	PSSE Executor	Yes	Text-based portable data file
Topology IR	`.json`	Topology Planner	Yes	Internal representation of network topology
Layout metadata	`.json`	Layout Engine	Yes	Component coordinates and orientations
Execution log	`.jsonl`	Logger	Yes	Complete structured execution trace
Validation report	`.json`	Validator	Yes	All validation check results
Execution summary	`.json`	Orchestrator	Yes	High-level workflow summary
PSSE diagram	`.sld`	PSSE Executor	Best-effort	Single-line diagram file
SVG diagram	`.svg`	Layout Engine	Fallback	Standalone visual diagram
Approval record	`.json`	HITL Controller	If HITL used	User approval/rejection with modifications
Retry report	`.json`	Recovery Engine	If retries occurred	All retry attempts and outcomes
Diagnostic report	`.md`	Orchestrator	On failure	Human-readable failure analysis

25.2 Folder Structure

```
{project_root}/
├── config/
│   ├── settings.yaml            # System configuration
│   ├── layout_config.yaml       # Layout parameters
│   └── templates/               # Topology templates
│       ├── single_bus.json
│       ├── double_bus.json
│       └── radial.json
├── runs/
│   └── {workflow_id}/           # One directory per execution
│       ├── input/
│       │   └── prompt.txt       # Raw user prompt
│       ├── topology/
│       │   ├── intent.json      # Parsed structured intent
│       │   ├── resolved_intent.json
│       │   ├── topology_ir_v1.json
│       │   └── topology_ir_v2.json  # If modified
│       ├── layout/
│       │   └── layout_map.json  # Component coordinates
│       ├── psse/
│       │   ├── model.sav        # PSSE saved case
│       │   ├── model.raw        # PSSE raw data
│       │   └── diagram.sld      # PSSE diagram (if generated)
│       ├── diagrams/
│       │   └── sld_export.svg   # Standalone diagram
│       ├── logs/
│       │   ├── execution.jsonl  # Full structured log
│       │   ├── summary.json     # Execution summary
│       │   └── timeline.json    # Event timeline
│       ├── reports/
│       │   ├── validation.json  # Validation results
│       │   ├── approval.json    # HITL approval record
│       │   ├── retry.json       # Retry attempts
│       │   └── diagnostic.md   # Failure diagnosis (if failed)
│       └── metadata.json        # Workflow metadata (state, timing)
└── logs/
    └── system.log               # System-wide log (all workflows)
```

25.3 Naming Conventions

- **workflow_id:** UUID v4, e.g., `a1b2c3d4-e5f6-7890-abcd-ef1234567890`
- **Shortened display ID:** First 8 characters, e.g., `a1b2c3d4`
- **Timestamp format:** ISO 8601 with timezone, e.g., `2026-05-22T18:30:00+05:30`
- **File naming:** Lowercase, underscores, no spaces

25.4 Reproducibility

Given: A `topology_ir_v{N}.json` file and `layout_map.json`
Guarantee: Re-running the PSSE Executor with these inputs produces an identical `.sav` file (byte-level may differ due to PSSE internals, but model content is identical).

Verification: Compare bus/branch/transformer data between original and reproduced models using `psspy.abusint()`, `psspy.abrnint()`, etc.

---

§26. UX + Interaction Design

26.1 POC UX: CLI Interface

```
$ python psse_agent.py

═══════════════════════════════════════════════════
  PSSE36 Agentic Modeller v0.1 (POC)
═══════════════════════════════════════════════════

Enter your command (or 'quit' to exit):

> Create a 33kV double bus substation with 6 feeders and bus coupler

── PARSING ──────────────────────────────────────
✓ Intent extracted (confidence: 0.92)

── ASSUMPTIONS ──────────────────────────────────
The following assumptions were made:
  • Topology: double_bus (confidence: 1.0)
  • Feeders: 6 (confidence: 1.0)
  • Bus coupler: yes (confidence: 1.0)
  • Transformers: 0 (not specified)
  • CT/PT: not included
  • Breaker arrangement: standard isolator-breaker-isolator
  • Bus numbering: starting at 100
  • Base MVA: 100
  • Feeder load: 10 MW each (placeholder)

── TOPOLOGY ─────────────────────────────────────
Constructed topology:
  • Buses: 9 (2 main + 1 coupler + 6 feeder)
  • Branches: 8 (1 coupler + 6 breakers + 1 tie)
  • Voltage levels: [33 kV]

── VALIDATION ───────────────────────────────────
✓ no_orphan_buses ..............  PASS
✓ no_floating_transformers .....  PASS (no transformers)
✓ voltage_consistency ..........  PASS
✓ no_duplicate_bus_numbers .....  PASS
✓ graph_connectivity ...........  PASS
⚠ source_exists ................  WARNING: No swing bus defined
✓ valid_bus_types ..............  PASS
✓ breaker_termination ..........  PASS
✓ no_self_loops ................  PASS
✓ unique_branch_ids ............  PASS

Results: 9 passed, 1 warning, 0 errors

── APPROVAL ─────────────────────────────────────
Proceed with PSSE execution? [Y]es / [N]o / [M]odify: Y

── EXECUTING ────────────────────────────────────
Creating bus 100 (MAIN-BUS-1, 33kV) .......... ✓
Creating bus 101 (MAIN-BUS-2, 33kV) .......... ✓
Creating bus 102 (COUPLER, 33kV) .............. ✓
Creating bus 110 (FEEDER-1, 33kV) ............. ✓
Creating bus 111 (FEEDER-2, 33kV) ............. ✓
Creating bus 112 (FEEDER-3, 33kV) ............. ✓
Creating bus 113 (FEEDER-4, 33kV) ............. ✓
Creating bus 114 (FEEDER-5, 33kV) ............. ✓
Creating bus 115 (FEEDER-6, 33kV) ............. ✓
Creating branch 100→102 (bus_coupler) ......... ✓
Creating branch 102→101 (bus_coupler) ......... ✓
Creating branch 100→110 (breaker) ............. ✓
Creating branch 100→111 (breaker) ............. ✓
Creating branch 100→112 (breaker) ............. ✓
Creating branch 101→113 (breaker) ............. ✓
```

```
Creating branch 101→114 (breaker) ..... ✓
Creating branch 101→115 (breaker) ..... ✓
Placing on SLD ........................ ✓
Saving model.sav ...................... ✓
Exporting model.raw ................... ✓

── COMPLETE ──────────────────────────────────────
✓ Workflow completed successfully

Artifacts:
  📁 runs/a1b2c3d4/psse/model.sav
  📁 runs/a1b2c3d4/psse/model.raw
  📁 runs/a1b2c3d4/topology/topology_ir_v1.json
  📁 runs/a1b2c3d4/diagrams/sld_export.svg
  📁 runs/a1b2c3d4/logs/execution.jsonl

Total time: 34.2 seconds
```

26.2 Error Handling UX

```
── EXECUTING ─────────────────────────────────────
Creating bus 100 (MAIN-BUS-1, 33kV) .......... ✓
Creating bus 101 (MAIN-BUS-2, 33kV) .......... ✓
Creating branch 100→101 (bus_coupler) ......... ✗ ERROR

⚠ PSSE API Error:
  Function: psspy.branch_data_3
  Error code: 3
  Likely cause: Invalid branch parameters

  Attempting repair (1/3)...
  Strategy: Adjusting impedance to minimum default

Creating branch 100→101 (bus_coupler) ......... ✓ (retry succeeded)
```

26.3 Production UX Evolution

Phase	Interface	Features
POC	CLI (typer/click)	Text prompt, structured output, Y/N approval
v1.0	Desktop app (PyQt6)	Chat panel, topology graph viewer, progress bar
v1.5	PSSE plugin	Embedded panel in PSSE GUI, direct model interaction
v2.0	Web UI	Browser-based interface, team features

---

§27. Tech Stack

27.1 Recommended Stack (Opinionated)

Component	Choice	Rationale	Alternatives Rejected
Language	Python 3.9	PSSE36 Python bridge requirement. Ecosystem maturity.	Python 3.11+ (PSSE compatibility risk)
Agent framework	Custom orchestration (no framework)	Pipeline is sequential and simple. LangGraph/AutoGen add dependency overhead and abstraction for a 6-agent linear pipeline. Build custom.	LangGraph (overhead for simple pipeline), AutoGen (multi-agent swarm focus, wrong paradigm), CrewAI (too opinionated)
LLM provider	OpenAI GPT-4o via API	Best structured output support (JSON mode). Reliable. Well-documented.	Anthropic Claude (good but JSON mode less mature), Local LLMs (quality insufficient for reliable extraction)
Graph library	NetworkX	Mature, well-documented, pure Python, perfect for graphs <1000 nodes.	igraph (C dependency, harder install), graph-tool (Linux-focused), custom (unnecessary)
Data validation	Pydantic v2	Type-safe models, JSON serialization, validation. Industry standard.	dataclasses (no validation), attrs (less ecosystem support)
CLI framework	Typer	Clean API, auto-help, type hints. Built on Click.	Click (more verbose), argparse (too primitive)
IR format	JSON with JSON Schema validation	Human-readable, debuggable, standard tooling.	YAML (whitespace-sensitive errors), Protobuf (overkill, binary), XML (verbose)
Logging	Python `logging` + `structlog`	Structured JSON logging with minimal setup.	loguru (less structured), custom (unnecessary)
Storage	SQLite (via `sqlite3`)	Zero-config, single-file database, perfect for local tool.	PostgreSQL (requires server, overkill), flat files (no query capability)
Windows automation	pywinauto	Mature Windows GUI automation. Only if needed for diagram placement.	pyautogui (less Windows-aware), win32com (lower-level)
HTTP client	httpx	Modern async-capable HTTP client for LLM API calls.	requests (no async), aiohttp (more complex)
Testing	pytest + pytest-cov	Standard Python testing.	unittest (verbose), nose2 (less maintained)
Config	PyYAML + Pydantic settings	YAML config files validated by Pydantic.	TOML (fine but YAML more common for config), .env only (too limited)
SVG generation	svgwrite	Lightweight SVG diagram export.	matplotlib (heavy for diagrams), plotly (overkill)
Process monitoring	psutil	Monitor PSSE process health.	Manual process checks

27.2 Dependency Budget

```
# Core (required)
networkx>=3.0          # Graph operations
pydantic>=2.0          # Data models
typer>=0.9             # CLI
structlog>=23.0        # Structured logging
httpx>=0.24            # LLM API calls
pyyaml>=6.0            # Config files
svgwrite>=1.4          # SVG diagram export
psutil>=5.9            # Process monitoring
jsonschema>=4.0        # IR validation

# Optional (installed when needed)
pywinauto>=0.6         # GUI automation fallback
pillow>=10.0           # Image processing (screenshot verification)

# Development
pytest>=7.0
pytest-cov>=4.0
black>=23.0
ruff>=0.1
mypy>=1.0

# Total direct dependencies: 9 core + 2 optional + 5 dev = 16
# Target: ≤15 core dependencies ✓ (at 9)
```

27.3 Key Technical Decisions

Decision 1: Custom Orchestration vs LangGraph

Choice: Custom orchestration.

LangGraph provides a state machine abstraction for LLM agent workflows. However: - Our pipeline is a **linear sequence** of 8 stages, not a dynamic graph - LangGraph's value is in complex routing, branching, and cycles — we have minimal branching - Adding LangGraph introduces a dependency that the team must learn, debug, and maintain - Custom orchestration for a linear pipeline is ~200 lines of Python - If the pipeline becomes more complex post-POC, LangGraph can be introduced then

```
# This is our entire "orchestration framework":
def run_pipeline(prompt: str, config: Config) -> ExecutionResult:
    session = ExecutionSession(prompt=prompt)
    try:
        session.state = "parsing"
        intent = parse_intent(prompt)

        session.state = "planning"
        graph, ir = plan_topology(intent)

        session.state = "validating"
        report = validate_topology(graph, ir)
        if report.has_errors:
            return ExecutionResult(success=False, error="Validation failed", report=report)

        session.state = "awaiting_approval"
        decision = request_approval(intent, ir, report)
        if decision.rejected:
            return ExecutionResult(success=False, error="User rejected")

        session.state = "laying_out"
        layout = compute_layout(graph, config.layout)

        session.state = "executing"
        exec_report = execute_in_psse(ir, layout, config.psse)

        session.state = "persisting"
        artifacts = persist_artifacts(session)

        session.state = "completed"
        return ExecutionResult(success=True, artifacts=artifacts)

    except Exception as e:
        session.state = "failed"
        log_failure(session, e)
        return ExecutionResult(success=False, error=str(e))
```

Decision 2: NetworkX vs Custom Graph

Choice: NetworkX.

- Battle-tested library with 15+ years of development
- Handles all our graph operations: construction, traversal, validation, serialization
- Our graphs are small (<1000 nodes) — performance is irrelevant
- Rich algorithm library (connectivity, shortest path, cycles) available if needed
- No reason to build custom graph primitives

Decision 3: SQLite vs PostgreSQL

Choice: SQLite.

- Zero configuration. No server process. Single file.
- Perfectly adequate for single-user, local tool
- Supports SQL queries for log analysis
- File-based — easy to backup, copy, share
- If multi-user is ever needed (unlikely for 12+ months), migrate to Postgres then

Decision 4: CLI-First vs GUI-First

Choice: CLI-first.

- Fastest to implement (days, not weeks)

- Perfectly adequate for POC validation
- Engineers are comfortable with command-line tools
- Easy to script and automate
- GUI can be added as a layer on top of the CLI core without architectural changes

---

§28. POC Roadmap

Milestone P0: Hello World — PSSE Connection

Attribute	Value
Objective	Prove that Python can programmatically create objects in PSSE36
Duration	3–5 days
Dependencies	PSSE36 installed, Python path configured
Features	Import psspy. Initialize PSSE. Create 1 bus. Save .sav file. Load .sav file. Verify bus exists.
Deliverable	Python script that creates a single bus in PSSE and saves a loadable .sav file
Acceptance criteria	1. `psspy.psseinit()` succeeds. 2. `psspy.bus_data_4()` creates bus without error. 3. `.sav` file loads in PSSE GUI showing 1 bus. 4. Script runs deterministically on repeated execution.
Risks	PSSE Python path misconfiguration. Python version mismatch. psspy import failures. PSSE license issues.
Demo	Screen recording: run script → open .sav in PSSE → bus visible
Engineering effort	1 engineer, 3–5 days
Complexity	Low

Milestone P1: Component Creation

Attribute	Value
Objective	Create all core PSSE objects programmatically
Duration	5–7 days
Dependencies	P0 complete
Features	Create buses (all types). Create branches. Create two-winding transformers. Create loads. Create machines. Save and verify each type. Handle API error codes.
Deliverable	Python module `psse_creator.py` with functions for each PSSE object type
Acceptance criteria	1. Can create a 5-bus radial network programmatically. 2. All objects visible in PSSE GUI. 3. Power flow converges on the created network. 4. Error codes properly handled and logged.
Risks	Undocumented API parameter quirks. Transformer parameter mapping complexity.
Demo	5-bus network created by script, power flow runs successfully
Engineering effort	1 engineer, 5–7 days
Complexity	Medium

Milestone P2: Topology Graph Engine

Attribute	Value
Objective	Build internal graph representation and convert to PSSE objects
Duration	5–7 days
Dependencies	P1 complete
Features	NetworkX graph construction. IR JSON schema. Template topologies (radial, single_bus). Graph → IR serialization. IR → PSSE execution. Ingestion of row-wise structured engineering schema (Excel/CSV/JSON).
Deliverable	`topology_engine.py` module with template builders. `ir_schema.json`. `psse_executor.py` that consumes IR.
Acceptance criteria	1. `build_radial(5, 33.0)` produces valid NetworkX graph. 2. Graph serializes to valid IR JSON. 3. IR JSON feeds into executor, producing valid PSSE model. 4. Round-trip: graph → IR → PSSE → verify matches graph.
Risks	IR schema may need iteration. Template complexity for double-bus.
Demo	Code demo: `build_radial()` → IR → PSSE model → power flow
Engineering effort	1 engineer, 5–7 days
Complexity	Medium

Milestone P3: Validation Engine

Attribute	Value
Objective	Detect invalid topologies before PSSE execution
Duration	3–5 days
Dependencies	P2 complete
Features	10 core validation checks (see §17). ValidationReport data model. Severity classification. Error/warning presentation.
Deliverable	`validator.py` module with check registry
Acceptance criteria	1. Correctly detects orphan buses. 2. Correctly detects floating transformers. 3. Correctly detects disconnected graphs. 4. Passes valid topologies without false positives. 5. ≥5 test cases per check.
Risks	Low. Deterministic graph checks are well-defined.
Demo	Feed intentionally broken topologies → validator catches errors
Engineering effort	1 engineer, 3–5 days
Complexity	Low–Medium

Milestone P4: Basic SLD Generation

Attribute	Value
Objective	Automatically place topology components on a visual diagram
Duration	7–10 days
Dependencies	P2 complete. P0 diagram API investigation complete.
Features	Hierarchical layout algorithm. Coordinate computation. PSSE diagram placement (or SVG fallback). Collision detection.
Deliverable	`layout_engine.py`. Either PSSE diagram integration or SVG export.
Acceptance criteria	1. For radial network ≤20 buses: zero overlapping components. 2. Busbars rendered horizontally. 3. Feeders rendered vertically. 4. Output viewable (in PSSE or as SVG). 5. Deterministic: same input → same layout.
Risks	HIGH. PSSE diagram API may not exist or may be poorly documented. SVG fallback is lower risk but less valuable.
Demo	Generated SLD image/screenshot showing readable topology layout
Engineering effort	1–2 engineers, 7–10 days
Complexity	High

Milestone P5: Natural Language Interface

Attribute	Value
Objective	Convert natural language prompts to validated structured engineering input (Excel/CSV/JSON format) mapping to IR
Duration	5–7 days
Dependencies	P2 complete. LLM API access.
Features	LLM prompt design. StructuredIntent Pydantic model. Few-shot examples. Deterministic pre-extraction (regex). Confidence scoring. Assumption generation.
Deliverable	`intent_parser.py`. System prompt + few-shot examples.
Acceptance criteria	1. Correctly extracts voltage from "33kV substation". 2. Correctly identifies "double bus" topology. 3. Correctly counts "6 feeders". 4. Confidence scores reflect extraction quality. 5. ≥80% accuracy on 20-prompt test suite.
Risks	LLM may hallucinate equipment. JSON parsing may fail. Prompt engineering iteration needed.
Demo	Type prompt → see extracted intent → see generated topology
Engineering effort	1 engineer, 5–7 days
Complexity	Medium

Milestone P6: Retry Engine

Attribute	Value
Objective	Self-correct on PSSE execution failures
Duration	3–5 days
Dependencies	P1, P2 complete
Features	Failure classification. Retry budget. Backoff logic. Parameter repair for common errors. Escalation to human.
Deliverable	`recovery_engine.py`. Retry budget configuration.
Acceptance criteria	1. Transient failures (simulated timeout) retried successfully. 2. Parameter errors (wrong impedance) auto-corrected. 3. Budget exhaustion triggers clean failure report. 4. No infinite retry loops.
Risks	Low. Simple switch/retry logic.
Demo	Inject failure → system retries → succeeds (or cleanly fails)
Engineering effort	1 engineer, 3–5 days
Complexity	Low

Milestone P7: Human Approval

Attribute	Value
Objective	Implement HITL checkpoint before PSSE execution
Duration	2–3 days
Dependencies	P5 complete
Features	CLI approval display. Assumption presentation. Approve/Reject/Modify flow. Approval logging.
Deliverable	`hitl_controller.py`. CLI approval prompts.
Acceptance criteria	1. Assumptions displayed before execution. 2. User can approve → execution proceeds. 3. User can reject → clean stop. 4. User can modify → re-parse with corrections. 5. All approvals logged.
Risks	Low. CLI UX only.
Demo	Full interactive demo: prompt → approval → execution → artifacts
Engineering effort	1 engineer, 2–3 days
Complexity	Low

Milestone P8: Robust POC

Attribute	Value
Objective	End-to-end stable demonstration: NL prompt → PSSE model + SLD
Duration	5–7 days
Dependencies	P0–P7 complete
Features	Full pipeline integration. CLI interface. Structured logging. Artifact persistence. At least 3 working topology templates. Error handling. 10+ golden test cases passing.
Deliverable	Installable Python package. README with quickstart. Demo video.
Acceptance criteria	1. "Create a radial network with 5 buses at 33kV" produces valid PSSE model. 2. "Create a 33kV single bus substation with 4 feeders" produces valid model with layout. 3. "Create a 33kV double bus substation with 6 feeders" produces valid model. 4. All 3 demos reproducible. 5. Full logs for each execution. 6. ≤2 manual retries needed across all 3 demos.
Risks	Integration issues between subsystems. PSSE diagram placement may require manual intervention.
Demo	Live demo: engineer types 3 different prompts, system generates 3 PSSE models end-to-end
Engineering effort	1–2 engineers, 5–7 days
Complexity	Medium (integration)

Roadmap Timeline Summary

```
Week 1-2:    P0 (PSSE connection) + P1 (component creation)
Week 2-3:    P2 (topology graph) + P3 (validation)
Week 3-5:    P4 (SLD generation) — HIGH RISK, may need more time
Week 4-5:    P5 (NL interface) — parallel with P4
Week 5-6:    P6 (retry) + P7 (HITL)
Week 6-7:    P8 (integration + testing)
Week 8:      Buffer + polish + documentation

Total estimated POC timeline: 7–10 weeks with 1–2 engineers
```

Input Modality Roadmap Sequencing

To establish a solid foundation before introducing natural language layers, the ingestion roadmap adheres to the following sequence: * **Stage A: Structured Engineering Schema Ingestion** (Direct ingestion of Excel/CSV/JSON-like tabular schema mapping to TopologyIR). Developed in parallel with Milestone P2. * **Stage B: Deterministic Topology Generation** (Processing validated structured schemas and templates into TopologyIR, validating constraints, and executing in PSSE). Developed in Milestones P1-P4. * **Stage C: Optional NLP Convenience Layer** (Converting natural language inputs to structured engineering schemas upstream of TopologyIR). Developed in Milestone P5.

This staging guarantees a functional, robust, and deterministic engineering core first before layer convenience interfaces are integrated.

---

§29. Engineering Backlog

Epic 1: PSSE Integration Foundation

Task	Priority	Difficulty	Risk	Depends On	Estimate	Output
Configure Python path for psspy import	P0	Low	Medium	PSSE installed	0.5 days	Working import
Create single bus via API	P0	Low	Low	psspy import	0.5 days	Bus creation function
Create branch via API	P0	Low	Low	Bus creation	0.5 days	Branch creation function
Create two-winding transformer via API	P0	Medium	Medium	Bus creation	1 day	Transformer function
Create load and machine via API	P0	Low	Low	Bus creation	0.5 days	Load/machine functions
Save/load .sav file	P0	Low	Low	Any object created	0.5 days	Save/load functions
Export .raw file	P0	Low	Low	Any object created	0.5 days	Raw export function
Investigate diagram API	P0	High	HIGH	psspy import	2 days	API capability report
Implement diagram placement	P1	High	HIGH	API investigation	3–5 days	Diagram placement
Error code handling	P0	Medium	Low	All API calls	1 day	Error handler

Epic 2: Topology Engine

Task	Priority	Difficulty	Risk	Depends On	Estimate	Output
Define IR JSON schema	P0	Medium	Low	None	1 day	Schema file
Implement Pydantic IR models	P0	Medium	Low	Schema	1 day	Models module
Build radial topology template	P0	Medium	Low	IR models	1 day	Template function
Build single-bus substation template	P1	Medium	Low	IR models	1.5 days	Template function
Build double-bus substation template	P1	Medium	Medium	IR models	2 days	Template function
IR serialization/deserialization	P0	Low	Low	IR models	0.5 days	Serde functions
Graph → IR → PSSE pipeline	P0	Medium	Medium	Templates + PSSE	2 days	Pipeline module
Template parameterization	P1	Medium	Low	Templates	1 day	Config system

Epic 3: Validation

Task	Priority	Difficulty	Risk	Depends On	Estimate	Output
Define ValidationResult model	P0	Low	Low	None	0.5 days	Model
Implement 10 core checks	P0	Medium	Low	Topology engine	2 days	Check functions
Build check registry	P0	Low	Low	Checks	0.5 days	Registry module
Test suite for checks (50+ cases)	P1	Medium	Low	Checks	2 days	Test file
Auto-fix implementation (3 fixers)	P2	Medium	Medium	Checks	1.5 days	Fixer functions

Epic 4: Layout Engine

Task	Priority	Difficulty	Risk	Depends On	Estimate	Output
Define LayoutConfig and LayoutMap models	P0	Low	Low	None	0.5 days	Models
Implement hierarchical layer assignment	P1	Medium	Low	Topology engine	1 day	Layer algorithm
Implement horizontal ordering	P1	Medium	Medium	Layer assignment	1 day	Ordering algorithm
Implement coordinate assignment	P1	Medium	Low	Ordering	1 day	Coordinate functions
Implement collision detection	P1	Medium	Medium	Coordinates	1.5 days	Collision detector
Implement SVG export	P1	Low	Low	Coordinates	1 day	SVG generator
Layout scoring function	P2	Medium	Low	Complete layout	1 day	Scoring function
Radial layout mode	P1	Low	Low	Coordinates	0.5 days	Mode handler
Double-bus layout mode	P1	High	Medium	Coordinates	2 days	Mode handler

Epic 5: NLP Interface

Task	Priority	Difficulty	Risk	Depends On	Estimate	Output
Design system prompt	P1	Medium	Medium	None	1 day	Prompt text
Create 10 few-shot examples	P1	Medium	Low	Prompt design	1 day	Example set
Implement StructuredIntent model	P0	Low	Low	None	0.5 days	Pydantic model
Implement LLM API caller	P1	Low	Low	httpx	0.5 days	API client
Implement response parser	P1	Medium	Medium	Intent model	1 day	Parser
Implement deterministic pre-extractor	P1	Low	Low	None	0.5 days	Regex module
Build 20-prompt test suite	P1	Medium	Low	Parser	1 day	Test file
Assumption engine	P1	Medium	Low	Intent model	1 day	Default resolver

Epic 6: Orchestration & Infrastructure

Task	Priority	Difficulty	Risk	Depends On	Estimate	Output
Implement pipeline runner	P1	Medium	Low	All agents	1.5 days	Orchestrator
Implement structured logger	P0	Low	Low	None	1 day	Logger module
Implement state machine	P1	Medium	Low	Pipeline runner	1 day	State module
Implement CLI interface	P1	Low	Low	Pipeline runner	1 day	CLI module
Implement HITL checkpoint	P1	Medium	Low	CLI	1 day	HITL module
Implement retry engine	P1	Medium	Low	Pipeline runner	1 day	Retry module
Implement artifact manager	P1	Low	Low	Folder structure	0.5 days	Artifact module
Session manager	P1	Low	Low	None	0.5 days	Session module

---

§30. Testing Strategy

30.1 Test Categories

Category	Scope	PSSE Required?	Count Target
Unit tests	Individual functions, models, validators	No	100+
Integration tests	Multi-module pipelines (graph→IR→validation)	No	30+
PSSE tests	API calls, object creation, file save/load	Yes	20+
Workflow tests	End-to-end prompt→model	Yes	10+
NLP tests	Intent extraction accuracy	No (mock LLM)	30+
Layout tests	Coordinate computation, collision detection	No	20+
Regression tests	Golden test cases that must never break	Partial	10+

30.2 Golden Test Cases

#	Input Prompt	Expected Outcome
G1	"Create a radial network with 3 buses at 33kV"	3 buses, 2 branches, all at 33kV, connected linearly
G2	"Create a radial network with 5 buses at 132kV"	5 buses, 4 branches, linear chain
G3	"Create a 33kV single bus substation with 4 feeders"	1 main bus + 4 feeder buses + 4 breaker branches
G4	"Create a 33kV double bus substation with 6 feeders"	2 main buses + 6 feeder buses + bus coupler
G5	"Create a 33kV double bus with bus coupler and 4 feeders"	2 main buses + coupler bus + 4 feeder buses
G6	"Create 132/33kV substation with 1 transformer"	2 voltage levels, transformer between them
G7	"Create a simple 2-bus system with a line between them"	2 buses, 1 line branch
G8	"Create a 33kV bus with a 10MW load"	1 bus + 1 load
G9	"Create a 5-bus ring network at 33kV"	5 buses, 5 branches forming a cycle
G10	"Create a solar evacuation substation connected to 132kV grid"	Grid bus + transformer + solar bus

30.3 Test Assertions

For each golden test:

```
def test_golden_case_G1():
    result = run_pipeline("Create a radial network with 3 buses at 33kV", config)
    ir = result.topology_ir

    # Topology correctness
    assert len(ir.buses) == 3
    assert len(ir.branches) == 2
    assert all(b.base_kv == 33.0 for b in ir.buses)

    # Graph connectivity
    graph = result.topology_graph
    assert nx.is_connected(graph.to_undirected())

    # Validation passes
    assert result.validation_report.passed
    assert result.validation_report.error_count == 0

    # Layout quality (if layout enabled)
    if result.layout_map:
        assert count_overlaps(result.layout_map) == 0
        assert all_within_canvas(result.layout_map)

    # PSSE file validity (if PSSE enabled)
    if result.artifacts.get("sav"):
        # Load and verify
        psspy.case(result.artifacts["sav"])
        ierr, bus_count = psspy.abuscount(-1, 1)
        assert bus_count == 3

    # Reproducibility
    result2 = run_pipeline("Create a radial network with 3 buses at 33kV", config)
    assert result.topology_ir == result2.topology_ir
```

30.4 NLP Test Suite

```
NLP_TEST_CASES = [
    {
        "prompt": "Create a 33kV substation with double bus arrangement and 6 outgoing feeders",
        "expected": {
            "voltage_levels_kv": [33.0],
            "topology_type": "double_bus",
            "equipment_list": [{"equipment_type": "feeder", "count": 6}]
        }
    },
    {
        "prompt": "5 bus radial system at 11kV",
        "expected": {
            "voltage_levels_kv": [11.0],
            "topology_type": "radial",
            "bus_count_override": 5
        }
    },
    {
        "prompt": "132/33kV transformation substation with 2 transformers",
        "expected": {
            "voltage_levels_kv": [132.0, 33.0],
            "equipment_list": [{"equipment_type": "transformer", "count": 2}]
        }
    },
    # ... 17 more cases
]

@pytest.mark.parametrize("case", NLP_TEST_CASES)
def test_nlp_extraction(case, mock_llm):
    result = parse_intent(case["prompt"])
    for key, expected_value in case["expected"].items():
        assert getattr(result, key) == expected_value, f"Failed for {key} in: {case['prompt']}"
```

30.5 Layout Tests

```
def test_no_overlaps_radial_20_buses():
    graph = build_radial(20, 33.0)
    layout = compute_layout(graph, LayoutConfig())
    overlaps = count_overlaps(layout)
    assert overlaps == 0, f"Found {overlaps} overlapping components"

def test_no_overlaps_double_bus_6_feeders():
    graph = build_double_bus(bus_start=100, voltage_kv=33.0, feeder_count=6)
    layout = compute_layout(graph, LayoutConfig())
    overlaps = count_overlaps(layout)
    assert overlaps == 0

def test_busbar_horizontal():
    graph = build_single_bus(bus_start=100, voltage_kv=33.0, feeder_count=4)
    layout = compute_layout(graph, LayoutConfig())
    main_buses = [c for c in layout if graph.nodes[c].get("category") == "main_bus"]
    for bus_id in main_buses:
        assert layout[bus_id].orientation == Orientation.HORIZONTAL

def test_deterministic_layout():
    graph = build_radial(10, 33.0)
    layout1 = compute_layout(graph, LayoutConfig())
    layout2 = compute_layout(graph, LayoutConfig())
    assert layout1 == layout2
```

30.6 Fuzz Testing for Prompts

```
import random

FUZZ_VOCABULARY = {
    "voltages": ["11kV", "33kV", "66kV", "132kV", "220kV", "400kV"],
    "topologies": ["radial", "single bus", "double bus", "ring bus"],
    "equipment": ["transformer", "breaker", "feeder", "capacitor", "reactor"],
    "counts": ["1", "2", "3", "4", "5", "6", "8", "10"],
    "templates": [
        "Create a {voltage} {topology} substation with {count} {equipment}",
        "{count} bus {topology} network at {voltage}",
        "Generate a {voltage} substation with {topology} arrangement",
    ]
}

def generate_fuzz_prompt() -> str:
    template = random.choice(FUZZ_VOCABULARY["templates"])
    return template.format(
        voltage=random.choice(FUZZ_VOCABULARY["voltages"]),
        topology=random.choice(FUZZ_VOCABULARY["topologies"]),
        count=random.choice(FUZZ_VOCABULARY["counts"]),
        equipment=random.choice(FUZZ_VOCABULARY["equipment"])
    )

@pytest.mark.parametrize("_", range(50))
def test_fuzz_no_crash(_):
    prompt = generate_fuzz_prompt()
    # System should never crash — may reject invalid prompts, but should not throw unhandled exceptions
    try:
        result = parse_intent(prompt)
        assert result is not None
    except UserFacingError:
        pass  # Expected for some fuzz inputs
    # No unhandled exception = pass
```

§31. Failure Modes

31.1 Comprehensive Failure Analysis

F01: Ambiguous Prompt

Attribute	Value
Root cause	User provides insufficient information (e.g., "create a substation")
Detection	Confidence scores < 0.5 on critical fields (voltage, topology_type)
Severity	Medium — blocks execution but no data corruption
Mitigation	Request clarification via HITL. Apply defaults with explicit warning.
POC strategy	Apply defaults aggressively, log all assumptions
Production strategy	Interactive clarification dialog. Contextual suggestions. Learning from past prompts.

F02: Hallucinated Topology

Attribute	Value
Root cause	LLM invents equipment or connections not mentioned in prompt
Detection	Cross-reference LLM output against regex pre-extraction. Validate extracted equipment against domain vocabulary.
Severity	High — produces incorrect model without user awareness
Mitigation	Deterministic pre-extraction as validation anchor. HITL checkpoint. Equipment type whitelist.
POC strategy	Strict equipment type enum. Reject unknown types. Display all extracted fields for approval.
Production strategy	Domain ontology validation. Confidence-weighted extraction. Multi-pass verification.

F03: Invalid Assumptions

Attribute	Value
Root cause	System applies default that contradicts user intent
Detection	HITL checkpoint — user reviews and rejects
Severity	Medium — caught by HITL if engineer is attentive
Mitigation	All assumptions explicitly logged and displayed. Conservative defaults.
POC strategy	Display all assumptions prominently before execution
Production strategy	Learned defaults per user/project. Configurable assumption policy.

F04: PSSE API Mismatch

Attribute	Value
Root cause	psspy function signature differs from documentation, or undocumented parameter behavior
Detection	Non-zero return code from psspy call
Severity	High — blocks execution
Mitigation	Wrap every API call with error handling. Log exact parameters. Build parameter mapping through systematic testing in P0/P1.
POC strategy	Extensive P0/P1 testing to map all parameter quirks. Hard-code working parameter sets.
Production strategy	Parameter validation layer before API call. Known-good parameter database.

F05: GUI Automation Failure

Attribute	Value
Root cause	pywinauto cannot find expected UI element (version change, resolution, locale)
Detection	ElementNotFoundError or timeout
Severity	Medium — only affects diagram placement, not model creation
Mitigation	Use API first. GUI automation only as fallback. Version-lock UI element identifiers.
POC strategy	Accept diagram failure gracefully. Model creation is the primary deliverable.
Production strategy	UI element discovery with fuzzy matching. Screenshot verification. Multiple automation strategies.

F06: Diagram Overlap

Attribute	Value
Root cause	Layout algorithm fails to prevent component overlap
Detection	Collision detection in layout scoring
Severity	Low — diagram is ugly but model is correct
Mitigation	Increase spacing. Retry with different layout parameters. Manual adjustment acceptable for POC.
POC strategy	Log overlap count. Accept ≤2 overlaps for complex topologies.
Production strategy	Constraint solver for collision resolution. Multiple layout strategies.

F07: Floating Equipment

Attribute	Value
Root cause	Transformer or breaker created without proper bus connections
Detection	Validation check `no_floating_transformers`, `breaker_termination`
Severity	High — model is electrically invalid
Mitigation	Validation catches this before PSSE execution. Graph connectivity check.
POC strategy	Validate before execute. Block on ERROR-severity validation failures.
Production strategy	Auto-fix: connect floating equipment to nearest appropriate bus.

F08: Invalid Graph (Disconnected)

Attribute	Value
Root cause	Topology planner creates disconnected subgraphs
Detection	`nx.is_connected(graph.to_undirected())` returns False
Severity	High — PSSE may create islanded network
Mitigation	Graph connectivity check in validation.
POC strategy	Block execution on disconnected graph.
Production strategy	Auto-connect: add tie line between disconnected components.

F09: Corrupted Save File

Attribute	Value
Root cause	PSSE crashes during save, disk full, permission error
Detection	Non-zero return from `psspy.save()`. File size check (0 bytes = corrupt).
Severity	High — lost work
Mitigation	Save to temp file first, then rename. Verify file after save. Keep previous version.
POC strategy	Check return code. Log error. Re-save once.
Production strategy	Transactional save: write to temp, verify, rename. Backup previous version.

F10: Partial Execution

Attribute	Value
Root cause	Execution fails midway — some buses created, others not
Detection	Execution report shows partial completion
Severity	Medium — model is incomplete
Mitigation	Start from clean case (newcase_2). Re-execute entire IR.
POC strategy	Always start fresh. Never modify existing case.
Production strategy	Checkpoint after each phase. Rollback to checkpoint on failure.

F11: Duplicate Identifiers

Attribute	Value
Root cause	Two buses assigned same number, or branch with same (from, to, ckt)
Detection	Validation check `no_duplicate_bus_numbers`, `unique_branch_ids`
Severity	High — PSSE will overwrite or error
Mitigation	Deterministic numbering scheme. Validation before execution.
POC strategy	Sequential numbering from configurable start. Validate uniqueness.
Production strategy	Central bus number registry. Conflict resolution.

F12: PSSE Process Crash

Attribute	Value
Root cause	PSSE internal error causes process termination
Detection	psutil process monitoring. Exception from psspy.
Severity	High — loses all in-memory state
Mitigation	IR is persistent (saved to JSON). Replay from IR after restart.
POC strategy	Log crash. Report to user. User manually restarts.
Production strategy	Auto-restart PSSE subprocess. Auto-replay from last checkpoint.

F13: Windows Instability

Attribute	Value
Root cause	OS-level issues: memory pressure, PSSE window focus loss, DPI scaling
Detection	Process monitoring. System resource checks.
Severity	Low–Medium — usually recoverable
Mitigation	Monitor system resources before execution. Warn if low memory.
POC strategy	Ensure adequate system resources documented in README.
Production strategy	Pre-flight checks: memory, disk space, PSSE process health.

F14: User Rejection Loop

Attribute	Value
Root cause	User repeatedly rejects assumptions, system cannot satisfy
Detection	Rejection count > 3 for same prompt
Severity	Low — no system damage, just frustration
Mitigation	After 2 rejections, suggest user provide more specific prompt. Offer structured input form.
POC strategy	Log rejections. Suggest rephrasing after 2 rejections.
Production strategy	Learn from rejections. Adaptive defaults. Structured input fallback.

F15: Retry Loop Explosion

Attribute	Value
Root cause	System retries indefinitely due to persistent underlying error
Detection	Retry count exceeds budget
Severity	Medium — wastes time, may corrupt PSSE state
Mitigation	Hard retry budget. Exponential backoff. Different strategy per retry.
POC strategy	Max 3 retries per operation, 8 per workflow. Hard stop.
Production strategy	Smart escalation: change strategy, then escalate to human.

F16: Silent Failures

Attribute	Value
Root cause	PSSE API returns success (0) but object is not correctly created
Detection	Post-execution verification: count objects, check connectivity
Severity	High — model appears correct but isn't
Mitigation	Verification stage checks bus count, branch count, connectivity.
POC strategy	Verify object counts match IR. Attempt power flow.
Production strategy	Deep verification: read back all object parameters, compare to IR.

§32. Risk Register

ID	Risk	Likelihood	Impact	Severity	Mitigation	Owner
R1	PSSE diagram API does not exist	High	High	**Critical**	Fallback to SVG export + GUI automation. Diagram is secondary to model.	Tech Lead
R2	psspy parameter mapping is undocumented	Medium	High	**High**	Systematic P0/P1 testing. Community resources. Siemens support.	PSSE Specialist
R3	LLM extraction accuracy insufficient	Medium	Medium	**Medium**	More few-shot examples. Deterministic pre-extraction. HITL catches errors.	AI Engineer
R4	PSSE Python version incompatibility	Low	High	**Medium**	Verify Python version at P0. Pin to compatible version.	DevOps
R5	Layout algorithm produces poor diagrams	Medium	Medium	**Medium**	Iterative improvement. Acceptable for POC. Manual cleanup option.	Layout Engineer
R6	PSSE crashes frequently during automation	Low	Medium	**Medium**	Crash recovery. IR replay. Subprocess isolation in production.	PSSE Specialist
R7	Scope creep beyond POC boundaries	High	Medium	**Medium**	Strict scope freeze. Defer non-essential features.	Engineering Manager
R8	Team unfamiliar with PSSE API	Medium	Medium	**Medium**	Dedicated P0 exploration time. Documentation.	Team
R9	LLM API costs accumulate	Low	Low	**Low**	Cache responses. Limit to 1 LLM call per workflow. Local models later.	Engineering Manager
R10	Complex topologies not covered by templates	Medium	Medium	**Medium**	Start with 3 templates. Add incrementally. Accept manual specification.	Power Systems Engineer

§33. KPI Framework

33.1 Core Metrics

M1: Topology Correctness (%)

Definition: Fraction of generated topologies that pass all ERROR-severity validation checks.

Formula: `topology_correctness = (workflows_with_zero_validation_errors / total_workflows) × 100`

Measurement: Automated — validation engine produces this metric per workflow.

Target: POC ≥ 80%, Production ≥ 95%

M2: Layout Readability Score

Definition: Composite score (0.0–1.0) measuring SLD quality.

Formula: See §19.5 layout scoring function. Weighted combination of: - Overlap count (weight: 0.3) - Line crossing count (weight: 0.2) - Spacing uniformity (weight: 0.2) - Symmetry (weight: 0.15) - Alignment (weight: 0.15)

Measurement: Automated — layout engine computes score.

Target: POC ≥ 0.5, Production ≥ 0.8

M3: Execution Success Rate (%)

Definition: Fraction of workflows that complete the full pipeline (prompt → saved artifacts) without human intervention.

Formula: `success_rate = (workflows_completed / workflows_attempted) × 100`

Measurement: Automated — orchestrator logs completion/failure.

Target: POC ≥ 60% (simple topologies), Production ≥ 90%

M4: Retry Success Rate (%)

Definition: Fraction of failed operations that succeed after retry.

Formula: `retry_success = (retries_that_succeeded / total_retries) × 100`

Measurement: Automated — recovery engine logs.

Target: POC ≥ 30%, Production ≥ 60%

M5: Prompt Interpretation Accuracy (%)

Definition: Fraction of intent extraction fields that match ground truth.

Formula: `accuracy = (correctly_extracted_fields / total_fields) × 100` across test suite.

Measurement: Against labeled test suite of 20+ prompts.

Target: POC ≥ 80%, Production ≥ 92%

M6: Human Correction Frequency

Definition: Average number of user modifications per workflow.

Formula: `correction_freq = total_modifications / total_workflows`

Measurement: HITL checkpoint logs.

Target: POC ≤ 1.5, Production ≤ 0.3

M7: Diagram Overlap Count

Definition: Average number of overlapping component pairs per generated SLD.

Measurement: Collision detection in layout engine.

Target: POC ≤ 3, Production = 0

M8: End-to-End Time (seconds)

Definition: Wall-clock time from prompt submission to artifact persistence.

Measurement: Orchestrator timing logs.

Target: POC ≤ 180s (≤50 buses), Production ≤ 60s (≤200 buses)

M9: PSSE File Validity (%)

Definition: Fraction of generated .sav files that load in PSSE without errors.

Measurement: Automated — load and verify in PSSE.

Target: POC ≥ 90%, Production ≥ 99%

M10: Model Reproducibility (%)

Definition: Fraction of workflows that produce identical IR when re-run with same prompt and config.

Measurement: Run each golden test twice, compare IR hashes.

Target: 100% (deterministic by design, with cached LLM responses)

33.2 Benchmark Dataset

Create and maintain a benchmark dataset of 30+ labeled test cases: - 10 simple (radial, ≤5 buses) - 10 moderate (single/double bus substation, ≤20 buses) - 10 complex (multi-voltage, ≤50 buses)

Each case includes: prompt, expected StructuredIntent, expected bus count, expected branch count, expected validation result, expected layout properties.

33.3 Measurement Cadence

Metric	POC Cadence	Production Cadence
M1–M4	Per milestone	Per release + weekly
M5	Per NLP change	Per prompt engineering change
M6–M7	Per demo	Daily aggregate
M8	Per run	Per run (dashboard)
M9–M10	Per milestone	Per release (CI/CD)

§34. Production Hardening Roadmap

34.1 Maturity Model

```
LEVEL 0: POC (Target: 7-10 weeks)
├── Single-user local execution
├── CLI interface
├── 3 topology templates
├── Heuristic layout
├── Basic retry (3 attempts)
├── JSON file logging
├── Manual HITL via CLI
└── ≥60% success rate on simple topologies

LEVEL 1: STABLE TOOL (Target: +3-6 months)
├── 6+ topology templates
├── Layout scoring with collision avoidance
├── Structured SQLite logging
├── Configuration file system
├── 20+ golden tests passing in CI
├── Error diagnostics with suggested fixes
├── ≥80% success rate on moderate topologies
└── Documentation + quickstart guide

LEVEL 2: TEAM TOOL (Target: +6-12 months)
├── Desktop GUI (PyQt6)
├── Template library with custom templates
├── Layout quality ≥0.7 average score
├── Topology composition (combine substation + transmission)
├── Partial PDF interpretation (Level 2 automation)
├── User preference learning
├── ≥90% success rate
└── Team deployment guide

LEVEL 3: ENTERPRISE (Target: +12-24 months)
├── PSSE plugin integration
├── Multi-user with role-based approval
├── Audit trail for regulatory compliance
├── Advanced layout engine (constraint solver)
├── Study workflow integration (power flow, fault)
├── API for programmatic access
├── ≥95% success rate
└── Enterprise deployment package
```

34.2 Hardening Areas

Auditability

- **POC:** JSON log files per workflow
- **Level 1:** SQLite with indexed queries. Log viewer CLI tool.
- **Level 2:** Execution timeline visualization. Diff between IR versions.
- **Level 3:** Immutable audit log. Regulatory compliance reporting. Tamper detection.

Versioning

- **POC:** IR version string in metadata
- **Level 1:** Semantic versioning for IR schema. Migration scripts for schema changes.
- **Level 2:** Template versioning. Config versioning. Full dependency pinning.
- **Level 3:** Git-backed configuration. Change tracking across all artifacts.

HITL Maturity

- **POC:** CLI approve/reject/modify
- **Level 1:** Structured modification forms. Contextual suggestions.
- **Level 2:** Visual topology graph in approval UI. Interactive editing.
- **Level 3:** Role-based approval chains. Approval delegation. Policy engine.

Rollback

- **POC:** Re-run from scratch (newcase_2)
- **Level 1:** Checkpoint-based rollback within execution
- **Level 2:** Undo to any previous workflow state
- **Level 3:** Full transaction model with commit/rollback semantics

Verification Depth

- **POC:** Bus count check + power flow attempt
- **Level 1:** Full parameter readback and comparison to IR
- **Level 2:** Automated topology diagram comparison (generated vs expected)
- **Level 3:** Integration with analysis workflows (power flow convergence, fault levels)

Security

- **POC:** API keys in environment variables
- **Level 1:** Local secrets file with file permissions
- **Level 2:** No network exposure. Local execution only. Input sanitization.
- **Level 3:** Enterprise auth integration. Network isolation. Key rotation.

---

§35. Open Questions

35.1 Critical (Must Resolve in P0)

#	Question	Impact	Proposed Investigation
OQ-1	Does PSSE36 have a programmatic diagram/SLD API?	Determines entire diagram strategy	List all psspy functions. Check for `diag` / `sld` / `draw` / `place` functions. Inspect .sld file format.
OQ-2	What Python version does PSSE36's psspy require?	Determines Python version for entire project	Check PSSE36 installation. Test import with Python 3.8, 3.9, 3.10.
OQ-3	Can .sld diagram files be edited as text/XML?	Alternative to API for diagram creation	Open .sld file in text editor. Analyze format. Test manual edits.
OQ-4	What are the exact psspy function signatures for bus/branch/transformer creation?	Core functionality	Run `help(psspy.bus_data_4)`. Test with known-good parameters. Document return codes.

35.2 Important (Must Resolve in P1-P2)

#	Question	Impact
OQ-5	How does PSSE handle zero-impedance branches for breakers/isolators?	Breaker modelling strategy
OQ-6	What is the coordinate system for PSSE diagrams (pixels? units? range?)	Layout engine parameterization
OQ-7	Can PSSE diagrams be exported as image programmatically?	Verification strategy
OQ-8	How to suppress PSSE console output during automation?	Clean logging
OQ-9	What happens when psspy function is called with existing bus number?	Idempotency behavior

35.3 Deferred (Resolve When Needed)

#	Question	Impact
OQ-10	Is three-winding transformer modelling needed?	Template complexity
OQ-11	Should the system support PSSE35 or only PSSE36?	Compatibility scope
OQ-12	What LLM model provides best extraction accuracy for this domain?	NLP quality
OQ-13	Should GUI be PyQt6 or Electron?	Level 2 UX

---

§36. Implementation Recommendation

36.1 Team Composition (Ideal Small Team)

Role	Count	Responsibilities
Lead Engineer / Architect	1	Architecture decisions. PSSE integration. Pipeline orchestration. Code review.
Power Systems Engineer	1 (part-time)	Domain expertise. Topology template design. Validation rule definition. Acceptance testing.
AI/NLP Engineer	1	LLM prompt engineering. Intent extraction. Confidence scoring. NLP testing.
Total	2 full-time + 1 part-time	

Minimum viable team: 1 full-stack engineer with power systems knowledge + LLM access. Possible but slower.

36.2 Development Principles

1. **Ship daily.** Every day should produce a testable increment.
2. **Test first.** Write the golden test case before the implementation.
3. **Log everything.** When debugging fails, logs are the only source of truth.
4. **Fail loud.** Never catch and ignore exceptions. Every error must be visible.

5. **IR is sacred.** Never bypass the IR. Never go directly from prompt to PSSE.
6. **PSSE is fragile.** Treat every PSSE API call as potentially dangerous. Wrap, log, retry.
7. **Don't optimize prematurely.** The system will be fast enough if it works at all.
8. **Cut scope ruthlessly.** If a feature isn't needed for the demo, defer it.

---

§37. Suggested Development Order

Week 1: Foundation

```
Day 1-2: Project setup
  - Create Python project structure
  - Install dependencies (networkx, pydantic, typer, structlog)
  - Configure PSSE Python path
  - Verify psspy import works
  - Document Python version and PSSE version

Day 3-4: PSSE Hello World (P0)
  - Create single bus via psspy
  - Create 2-bus system with branch
  - Save and load .sav file
  - Document all API quirks discovered
  - Investigate diagram API (OQ-1, OQ-3)

Day 5: IR Foundation
  - Define Pydantic models for IR
  - Define JSON schema
  - Serialize/deserialize test
```

> *MVP Evolution Note: The validated MVP ingestion path now includes a structured engineering input adapter (Excel / CSV) as a dependency-free module that can be developed in parallel with PSSE foundation work.*

Week 2: Core Pipeline

```
Day 6-7: Component Creation (P1)
  - Implement create_bus, create_branch, create_transformer
  - Implement create_load, create_machine
  - Implement save/export functions
  - Handle error codes
  - Create 5-bus radial network test

Day 8-9: Topology Engine (P2)
  - Implement radial topology template
  - Implement single-bus substation template
  - Implement graph → IR serialization
  - Implement IR → PSSE execution pipeline

Day 10: Validation (P3)
  - Implement 10 core validation checks
  - Test with valid and invalid topologies
```

Week 3: Visualization + NLP

```
Day 11-13: Layout Engine (P4)
  - Implement hierarchical layer assignment
  - Implement coordinate computation
  - Implement collision detection
  - Implement SVG export (guaranteed output)
  - Attempt PSSE diagram placement (best-effort)

Day 14-15: Intent Parser (P5)
  - Design system prompt with few-shot examples
  - Implement LLM API caller
  - Implement StructuredIntent model
  - Implement deterministic pre-extraction
  - Test with 10-prompt suite
```

Week 4: Integration

```
Day 16: Double-bus template
  - Implement double-bus topology template
  - Layout for double-bus arrangement

Day 17: Retry + HITL (P6, P7)
  - Implement retry engine with budget
  - Implement CLI approval checkpoint

Day 18-19: Pipeline Integration (P8)
  - Wire all components together
  - Implement CLI interface
  - End-to-end testing with 3 golden cases

Day 20: Polish
  - Structured logging throughout
  - Artifact persistence
  - README and quickstart guide
  - Demo recording
```

Week 5+: Hardening (If Time Permits)

```
- Additional topology templates (ring bus, breaker-and-half)
- Layout quality improvements
- More NLP test cases
- Edge case handling
- Error message improvement
- Configuration system
```

---

§38. Final Technical Recommendations

38.1 Recommended First Milestone: Next 7 Days

Goal: PSSE connection proven + 5-bus radial network created programmatically + IR schema defined.

Deliverables: 1. Python script that creates a 5-bus radial network in PSSE36 and saves a loadable .sav file 2. Documented API quirks (parameter formats, return codes, Python version) 3. Diagram API investigation report (what exists, what doesn't) 4. Pydantic IR models for buses, branches, transformers, loads, generators 5. JSON IR for the 5-bus test case

Why this first: Everything else depends on PSSE integration working. If psspy doesn't work as expected, the entire project plan needs revision. Discover this in week 1, not week 5.

38.2 Recommended First Milestone: Next 30 Days

Goal: End-to-end POC: natural language prompt → PSSE model + diagram artifact for 3 topology types.

Deliverables: 1. Working CLI: user types prompt, system produces PSSE model 2. Three working topology templates: radial, single-bus, double-bus 3. Intent extraction from natural language (≥80% accuracy on 20-prompt test suite) 4. Topology validation (10 checks) 5. SLD layout with SVG export (PSSE diagram if API available) 6. HITL approval checkpoint 7. Structured logging 8. 10 golden test cases passing

38.3 Highest-Risk Subsystem

PSSE Diagram Placement (SLD API).

Reason: The existence and capability of PSSE36's diagram API is unknown. If no programmatic API exists, the team must either: - Fall back to GUI automation (fragile, slow to implement) - Generate standalone SVG diagrams (valuable but not integrated) - Edit .sld files directly (if format is discoverable)

Mitigation: Investigate in day 3-4 (P0). Design the system so that diagram placement is a separate, replaceable module. The model creation pipeline must work without diagram placement.

38.4 Simplest Path to First Working Demo

```
1. Import psspy                              (30 minutes)
2. Create 3 buses with psspy.bus_data_4()    (1 hour)
3. Create 2 branches with psspy.branch_data_3()  (1 hour)
4. Save .sav file                            (15 minutes)
5. Open in PSSE GUI                          (5 minutes)
6. Show power flow running                   (15 minutes)

Total: ~3 hours for absolute minimum demo
(No NLP, no graph abstraction, no layout — just "Python controls PSSE")
```

> *MVP Recommended Path: Create a sample Excel workbook with bus, branch, and load rows. Run it through the ingestion adapter → TopologyIR → executor pipeline. This provides a structured, repeatable first demo without any hardcoded intent or NLP dependency.*

38.5 Biggest Architectural Mistake to Avoid

Going directly from natural language prompt to PSSE API calls without an intermediate representation.

This is the single most damaging architectural shortcut. Without IR: - Cannot validate before execution - Cannot retry without re-parsing - Cannot debug what was planned vs what was executed - Cannot reproduce results - Cannot test parsing and execution independently - Cannot evolve the system incrementally

The IR is the spine of the architecture. Every subsystem reads from or writes to the IR. Skipping it saves 2 days of implementation and costs months of debugging.

38.6 Recommended MVP Scope Freeze

Freeze scope at:

> **"A CLI tool that accepts a structured engineering Excel workbook (or CSV) describing substations or simple networks (≤50 buses), generates a validated topology, creates a PSSE36 model, attempts SLD placement, and saves runnable .sav/.raw files — with full logging and human approval before execution."**

Three supported topology types: 1. Radial network (parameterized by bus count, voltage) 2. Single-bus substation (parameterized by feeder count, voltage) 3. Double-bus substation (parameterized by feeder count, voltage, bus coupler)

Everything else is post-MVP.

38.7 Recommended "Do NOT Build Yet" List

Feature	Why Not Yet
Desktop GUI	CLI is sufficient for POC validation. GUI adds weeks of effort.
Template editor	Hardcode 3 templates. Template authoring is a post-POC concern.
PDF/drawing interpreter	Level 3 automation. Requires OCR/vision pipeline. Massive scope.
Multi-substation composition	Complex graph algebra. Not needed for single-substation POC.
Three-winding transformers	Edge case. Two-winding covers 80%+ of use cases.
Protection study integration	Different workflow. Focus on model creation first.
Fine-tuned LLM	No training data exists yet. Collect correction data from usage first.
Cloud deployment	PSSE is local Windows. Cloud adds zero value.
Multi-user features	Single engineer is the user. No collaboration needed.
Plugin architecture	Premature. Build monolith first, extract abstractions later.
Automated testing in CI with PSSE	PSSE license + Windows CI is complex. Test IR pipeline in CI. Test PSSE locally.
Advanced layout engine (OR-tools)	Heuristic layout is sufficient for POC. Optimize after validating the pipeline works.

---

§38.8 Final Statement

This PRD specifies a system that is: - **Narrowly scoped** (PSSE36 only, model + SLD creation only) - **Architecturally sound** (topology-first, IR-driven, observable) - **Honestly assessed** (diagram API is the highest risk, layout is the hardest problem) - **Implementable** (3-person team, 7–10 weeks for POC) - **Evolvable** (every subsystem has a POC→production path)

The system's value proposition is clear: **reduce a 2–6 hour manual workflow to a 5–15 minute prompted workflow**, while producing standardized, validated, reproducible, and fully-audited PSSE models.

The recommended first action is: **import psspy and create a bus.** Everything else follows from that.

End of PRD — Document Version 1.0.0 — 2026-05-22